E-commerce Chatbot Source Code

Generated 2026-04-28

Included Files

```
README.md
chatbot_engine.py
chatbot_features.py
product_catalog.py
analytics.py
category_predictor.py
evaluation.py
app.py
chat.py
db.py
train.py
model.py
nltk_utils.py
intents.json
products.csv
orders.csv
evaluation_survey.csv
requirements.txt
```

README.md

E-commerce Chatbot

A Python chatbot for a CoffeeMugs e-commerce store. It can answer product questions, show available products, track orders, create orders, cancel orders, update delivery addresses, and let an admin update order status.

The project includes both a Tkinter GUI and a command-line chat interface.

Features

- Intent classification with PyTorch
- Product catalog lookup from ‘products.csv‘
- Content-based product recommendations using product attributes, browsing context, and cart/order context
- Product popularity analysis from saved database orders
- Simple market basket analysis for frequently bought product pairs from saved database orders
- Product category prediction using a small Naive Bayes ML model trained from ‘products.csv‘
- Order tracking and management through SQL Server
- CSV seeding from ‘orders.csv‘
- GUI chat app with admin login support
- CLI chat app for terminal use
- Training script for rebuilding ‘data.pth‘ from ‘intents.json‘

Requirements

- Python 3.10 or newer
- SQL Server
- Microsoft ODBC Driver 18 for SQL Server
- Python packages from ‘requirements.txt‘

Setup

Create and activate a virtual environment:

```bash
python3 -m venv .venv
source .venv/bin/activate
```

Install dependencies:

```bash
pip install -r requirements.txt
```

Download NLTK data:

```bash
python3 -c "import nltk; nltk.download(’punkt’); nltk.download(’wordnet’);
nltk.download(’omw-1.4’)"
```

Database Configuration

The app uses SQL Server through ‘pyodbc‘. Set ‘MSSQL_CONNECTION_STRING‘ before running the chatbot:

```bash
export MSSQL_CONNECTION_STRING="Server=127.0.0.1,1433;Database=ChatbotDb;User
Id=sa;Password=your_password;TrustServerCertificate=yes;Encrypt=yes"
```

If this environment variable is not set, the app uses the fallback connection string in ‘db.py‘.

On startup, the app creates the required ‘orders‘ and ‘users‘ tables if they do not already exist. It also seeds initial order data from ‘orders.csv‘ when the orders table is empty.

Default admin account:

```text
Username: admin
Password: admin123
```

Train the Model

Run training when ‘data.pth‘ is missing or after editing ‘intents.json‘:

```bash
```

```
python3 train.py
```

This creates or updates ‘data.pth‘, which is required by the GUI and CLI chatbot.

Run the GUI

```bash
python3 app.py
```

In the GUI, type ‘admin login‘ to sign in as an admin. Admin users can update order status.

Run the CLI

```bash
python3 chat.py
```

Type ‘quit‘, ‘exit‘, or ‘bye‘ to leave the CLI chat.

Recommendation examples:

```text
Recommend similar products
Recommend something like P001
Suggest products for my cart
Show me personalized recommendations
```

Analytics examples:

```text
Show product popularity
What are the top selling products?
Market basket analysis
Which products are frequently bought together?
```

Product category prediction examples:

```text
Predict category for P001
What category is P004?
Predict category for insulated travel mug
Classify product Classic Red Mug
```

Test Responses

Run the response test script:

```bash
python3 generate_response.py
```

The script checks sample queries from ‘intents.json‘ and asks you to mark each generated response as correct or wrong.

Evaluation And Experimentation

Run the automated evaluation script:

```bash
python evaluation.py
```

It reports:

- Chatbot intent accuracy and average prediction time
- Recommendation quality with Precision@3, hit rate, and mean reciprocal rank
- A comparison between chatbot behavior with and without the recommendation module
- User satisfaction survey averages from ‘evaluation_survey.csv‘
- Product category prediction accuracy

Project Structure

- ‘app.py‘ - Tkinter GUI entry point
- ‘chat.py‘ - command-line chatbot entry point
- ‘chatbot_engine.py‘ - intent model loading, prediction, and request routing helpers
- ‘chatbot_features.py‘ - shared chatbot feature responses for recommendations, analytics, and category prediction

- ‘product_catalog.py‘ - product CSV loading, formatting, and recommendation ranking
- ‘analytics.py‘ - product popularity and market basket analysis
- ‘category_predictor.py‘ - Naive Bayes product category prediction
- ‘db.py‘ - SQL Server setup and order/user database functions
- ‘model.py‘, ‘nltk_utils.py‘, ‘train.py‘ - PyTorch intent-classification training pipeline
- ‘evaluation.py‘, ‘evaluation_survey.csv‘ - automated evaluation and survey metrics
- ‘generate_response.py‘, ‘intent_response.py‘ - manual testing utilities
- ‘intents.json‘, ‘data.pth‘ - chatbot intent data and trained model
- ‘products.csv‘, ‘orders.csv‘ - product catalog and seed order records

Typical Workflow

1. Install dependencies.
2. Configure SQL Server with ‘MSSQL_CONNECTION_STRING‘.
3. Run ‘python3 train.py‘ if ‘data.pth‘ needs to be created.
4. Start the GUI with ‘python3 app.py‘ or the CLI with ‘python3 chat.py‘.

chatbot_engine.py

```
import json
import random

import torch

from model import NeuralNet
from nltk_utils import bag_of_words, tokenize


CONFIDENCE_THRESHOLD = 0.75
PRODUCT_QUESTION_TAGS = {"items", "product_availability", "sizes", "material"}


def get_device():
    return torch.device("cuda" if torch.cuda.is_available() else "cpu")


def load_intents(file_path="intents.json"):
    with open(file_path, "r", encoding="utf-8") as file:
        return json.load(file)


def is_cancel_order_request(user_input):
    normalized = user_input.lower()
    cancel_words = {"cancel", "delete", "remove"}
    return "order" in normalized and any(word in normalized for word in cancel_words)


def is_create_order_request(user_input):
    normalized = user_input.lower()
    create_words = {"create", "place", "make", "add", "buy"}
    return any(word in normalized for word in create_words) and (
        "order" in normalized or "product" in normalized
    )


def is_admin_login_request(user_input):
    return user_input.lower() in {"admin login", "login as admin", "admin"}


def is_admin_logout_request(user_input):
    return user_input.lower() in {"admin logout", "logout"}


def is_update_status_request(user_input):
    normalized = user_input.lower()
    status_words = {"status", "location", "shipped", "delivered", "processing"}
    update_words = {"update", "change", "set", "mark"}
    return (
        "order" in normalized
        and any(word in normalized for word in status_words)
        and any(word in normalized for word in update_words)
    )


def is_recommendation_request(user_input):
    normalized = user_input.lower()
    recommendation_words = {
        "recommend",
        "recommendation",
        "suggest",
        "suggestion",
        "similar",
        "personalized",
        "best match",
        "you think i should buy",
        "what should i buy",
    }
    return any(word in normalized for word in recommendation_words)


def is_product_popularity_request(user_input):
    normalized = user_input.lower()
    popularity_words = {
        "popular product",
        "popular products",
        "product popularity",
        "best selling",
        "best-selling",
```

```
        "top selling",
        "top products",
        "sales analysis",
        "analytics",
    }
    return any(word in normalized for word in popularity_words)


def is_market_basket_request(user_input):
    normalized = user_input.lower()
    basket_words = {
        "market basket",
        "apriori",
        "frequently bought together",
        "bought together",
        "product pairs",
        "basket analysis",
    }
    return any(word in normalized for word in basket_words)


def is_category_prediction_request(user_input):
    normalized = user_input.lower()
    category_words = {
        "predict category",
        "category prediction",
        "what category",
        "which category",
        "classify product",
        "classify this product",
        "product category",
    }
    return any(word in normalized for word in category_words)


class ChatbotEngine:
    def __init__(self, model, all_words, tags, intents, device=None):
        self.model = model
        self.all_words = all_words
        self.tags = tags
        self.intents = intents
        self.device = device or get_device()

    @classmethod
    def from_files(cls, model_path="data.pth", intents_path="intents.json"):
        device = get_device()
        data = torch.load(model_path, map_location=device)
        model = NeuralNet(
            data["input_size"],
            data["hidden_size"],
            data["output_size"],
        ).to(device)
        model.load_state_dict(data["model_state"])
        model.eval()
        return cls(
            model=model,
            all_words=data["all_words"],
            tags=data["tags"],
            intents=load_intents(intents_path),
            device=device,
        )

    def predict(self, sentence):
        tokens = tokenize(sentence)
        features = bag_of_words(tokens, self.all_words)
        features = features.reshape(1, features.shape[0])
        features = torch.from_numpy(features).to(self.device)

        with torch.no_grad():
            output = self.model(features)
            _, predicted = torch.max(output, dim=1)
            probs = torch.softmax(output, dim=1)

        tag = self.tags[predicted.item()]
        probability = probs[0][predicted.item()].item()
        return tag, probability

    def get_response_text(self, tag, context=None):
        for intent in self.intents["intents"]:
            if tag != intent["tag"]:
                continue
```

```
    if context:
        for response in intent["responses"]:
            if "{context}" in response:
                return response.format(context=context)

    response = random.choice(intent["responses"])
    if "{context}" in response:
        return response.format(context=tag)
    return response

return "I do not understand..."
```

chatbot_features.py

```
from analytics import (
    format_market_basket_analysis,
    format_product_popularity,
    market_basket_analysis,
    product_popularity,
)
from category_predictor import format_category_prediction, predict_category
from product_catalog import format_recommendation_list, recommend_products


def recommendation_message(user_input, viewed_product_ids=None, cart_product_ids=None):
    recommendations = recommend_products(
        query=user_input,
        viewed_product_ids=viewed_product_ids,
        cart_product_ids=cart_product_ids,
    )
    return format_recommendation_list(recommendations)


def product_popularity_message():
    return format_product_popularity(product_popularity())


def market_basket_message():
    return format_market_basket_analysis(market_basket_analysis())


def category_prediction_message(user_input):
    return format_category_prediction(predict_category(user_input))
```

product_catalog.py

```
import csv
import os
import re
from collections import Counter


PRODUCTS_CSV = "products.csv"
PRODUCT_ID_PATTERN = re.compile(r"\bP\d{3,}\b", re.IGNORECASE)


def load_products(csv_path=PRODUCTS_CSV):
    if not os.path.exists(csv_path):
        return []

    with open(csv_path, mode="r") as file:
        return list(csv.DictReader(file))


def get_product_by_id(product_id, csv_path=PRODUCTS_CSV):
    product_id = (product_id or "").strip().lower()
    for product in load_products(csv_path):
        if product["product_id"].lower() == product_id:
            return product
    return None


def find_product_reference(text, csv_path=PRODUCTS_CSV):
    normalized = (text or "").strip().lower()
    if not normalized:
        return None

    product_id_match = PRODUCT_ID_PATTERN.search(normalized)
    if product_id_match:
        product = get_product_by_id(product_id_match.group(0), csv_path)
        if product:
            return product

    for product in load_products(csv_path):
        name = product["name"].lower()
        if name in normalized or normalized in name:
            return product
    return None


def format_product(product):
    return (
        f"{product['product_id']} - {product['name']} | "
        f"{product['color']} | {product['size']} | "
        f"${product['price']} | Stock: {product['stock']}"
    )


def format_product_list(products):
    if not products:
        return "No products are available right now."
    return "\n".join(format_product(product) for product in products)


def _product_tokens(product):
    text = " ".join(
        [
            product.get("name", ""),
            product.get("category", ""),
            product.get("color", ""),
            product.get("size", ""),
            product.get("material", ""),
            product.get("description", ""),
        ]
    ).lower()
    return set(re.findall(r"[a-z0-9]+", text))


def _price(product):
    try:
        return float(product.get("price") or 0)
    except ValueError:
        return 0
```

```
def _stock(product):
    try:
        return int(product.get("stock") or 0)
    except ValueError:
        return 0


def _similarity_score(candidate, source):
    score = 0
    if candidate.get("category") == source.get("category"):
        score += 4
    if candidate.get("material") == source.get("material"):
        score += 2
    if candidate.get("color") == source.get("color"):
        score += 1.5
    if candidate.get("size") == source.get("size"):
        score += 1

    candidate_tokens = _product_tokens(candidate)
    source_tokens = _product_tokens(source)
    if candidate_tokens or source_tokens:
        overlap = len(candidate_tokens & source_tokens)
        union = len(candidate_tokens | source_tokens)
        score += 2 * (overlap / union)

    price_gap = abs(_price(candidate) - _price(source))
    score += max(0, 1 - (price_gap / 25))
    score += min(_stock(candidate), 20) / 20
    return score


def _preference_profile(products):
    profile = {
        "category": Counter(),
        "color": Counter(),
        "material": Counter(),
        "size": Counter(),
    }
    for product in products:
        for field in profile:
            value = product.get(field)
            if value:
                profile[field][value] += 1
    return profile


def _preference_score(candidate, profile):
    weights = {
        "category": 3,
        "material": 2,
        "color": 1.5,
        "size": 1,
    }
    return sum(
        weights[field] * profile[field][candidate.get(field, "")]
        for field in profile
    )


def recommend_products(
    query=None,
    viewed_product_ids=None,
    cart_product_ids=None,
    limit=3,
    csv_path=PRODUCTS_CSV,
):
    products = load_products(csv_path)
    if not products:
        return []

    product_by_id = {product["product_id"]: product for product in products}
    source_products = []
    explicit_source_ids = set()
    explicit_product = find_product_reference(query, csv_path)
    if explicit_product:
        source_products.append(explicit_product)
        explicit_source_ids.add(explicit_product["product_id"])

    cart_ids = set(cart_product_ids or [])
    if not explicit_product:
        for product_id in cart_product_ids or []:
            product = product_by_id.get(product_id)
```

```
            if product and product not in source_products:
                source_products.append(product)

        for product_id in viewed_product_ids or []:
            product = product_by_id.get(product_id)
            if product and product not in source_products:
                source_products.append(product)

    viewed_ids = set(viewed_product_ids or [])
    source_ids = explicit_source_ids
    if not explicit_product:
        source_ids |= cart_ids
    if not explicit_product and len(viewed_ids) < len(products):
        source_ids |= viewed_ids
    candidates = [product for product in products if product["product_id"] not in source_ids]

    if not candidates:
        candidates = products

    if not source_products:
        ranked = sorted(
            candidates,
            key=lambda product: (
                _stock(product),
                product.get("category") == "Gift Set",
                -_price(product),
            ),
            reverse=True,
        )
        return ranked[:limit]

    profile = _preference_profile(source_products)
    ranked = sorted(
        candidates,
        key=lambda product: (
            max(_similarity_score(product, source) for source in source_products)
            + _preference_score(product, profile),
            _stock(product),
        ),
        reverse=True,
    )
    return ranked[:limit]


def recommendation_reason(product, source_products=None):
    source_products = source_products or []
    if source_products:
        source = source_products[0]
        matching_fields = [
            field
            for field in ("category", "material", "color", "size")
            if product.get(field) == source.get(field)
        ]
        if matching_fields:
            return "matches your interest in " + ", ".join(matching_fields)
    if _stock(product) > 0:
        return "available now and fits common mug preferences"
    return "closest catalog match"


def format_recommendation_list(products):
    if not products:
        return "I do not have enough product data to make recommendations right now."
    lines = ["Recommended products:"]
    for product in products:
        lines.append(f"- {format_product(product)}")
    return "\n".join(lines)
```

analytics.py

```
import csv
from collections import Counter, defaultdict
from itertools import combinations


ORDERS_CSV = "orders.csv"


def load_csv_order_rows(csv_path=ORDERS_CSV):
    with open(csv_path, mode="r") as file:
        return list(csv.DictReader(file))


def load_database_order_rows():
    from db import get_connection

    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute(
            """
            SELECT
                order_number,
                status,
                current_location,
                expected_delivery_date,
                customer_address,
                customer_name,
                customer_email,
                product_id,
                product_name,
                quantity,
                unit_price,
                total_price
            FROM dbo.orders;
            """
        )
        rows = []
        for row in cursor.fetchall():
            rows.append(
                {
                    "order_number": row.order_number,
                    "status": row.status,
                    "current_location": row.current_location,
                    "expected_delivery_date": row.expected_delivery_date,
                    "customer_address": row.customer_address,
                    "customer_name": row.customer_name,
                    "customer_email": row.customer_email,
                    "product_id": row.product_id,
                    "product_name": row.product_name,
                    "quantity": row.quantity,
                    "unit_price": row.unit_price,
                    "total_price": row.total_price,
                }
            )
        return rows


def load_order_rows(csv_path=ORDERS_CSV, use_database=True):
    if use_database:
        try:
            return load_database_order_rows()
        except Exception:
            pass
    return load_csv_order_rows(csv_path)


def _quantity(row):
    try:
        return int(row.get("quantity") or 1)
    except ValueError:
        return 1


def _total_price(row):
    try:
        return float(row.get("total_price") or 0)
    except ValueError:
        return 0
```

```
def product_popularity(csv_path=ORDERS_CSV, limit=5, use_database=True):
    quantity_counter = Counter()
    revenue_counter = Counter()
    name_by_id = {}

    for row in load_order_rows(csv_path, use_database):
        if row.get("status", "").lower() == "cancelled":
            continue

        product_id = row.get("product_id")
        if not product_id:
            continue

        name_by_id[product_id] = row.get("product_name") or product_id
        quantity_counter[product_id] += _quantity(row)
        revenue_counter[product_id] += _total_price(row)

    return [
        {
            "product_id": product_id,
            "product_name": name_by_id.get(product_id, product_id),
            "units_sold": units_sold,
            "revenue": revenue_counter[product_id],
        }
        for product_id, units_sold in quantity_counter.most_common(limit)
    ]


def format_product_popularity(rows):
    if not rows:
        return "No product sales data is available yet."

    lines = ["Product popularity analysis from saved orders:"]
    for index, row in enumerate(rows, start=1):
        lines.append(
            f"{index}. {row['product_id']} - {row['product_name']} | "
            f"Units sold: {row['units_sold']} | Revenue: ${row['revenue']:.2f}"
        )
    return "\n".join(lines)


def market_basket_analysis(
    csv_path=ORDERS_CSV,
    min_support=2,
    limit=5,
    use_database=True,
):
    baskets = defaultdict(set)
    product_names = {}

    for row in load_order_rows(csv_path, use_database):
        if row.get("status", "").lower() == "cancelled":
            continue

        product_id = row.get("product_id")
        customer_email = row.get("customer_email")
        if not product_id or not customer_email:
            continue

        baskets[customer_email].add(product_id)
        product_names[product_id] = row.get("product_name") or product_id

    pair_counter = Counter()
    basket_count = 0
    for basket in baskets.values():
        if len(basket) < 2:
            continue
        basket_count += 1
        for pair in combinations(sorted(basket), 2):
            pair_counter[pair] += 1

    results = []
    for pair, support_count in pair_counter.most_common():
        if support_count < min_support:
            continue
        support = support_count / max(basket_count, 1)
        results.append(
            {
                "pair": pair,
                "names": tuple(product_names[product_id] for product_id in pair),
                "support_count": support_count,
```

```
                "support": support,
            }
        )
        if len(results) >= limit:
            break

    return results


def format_market_basket_analysis(rows):
    if not rows:
        return "No frequent product pairs were found yet."

    lines = ["Market basket analysis from saved orders:"]
    for row in rows:
        left, right = row["pair"]
        left_name, right_name = row["names"]
        lines.append(
            f"- {left} ({left_name}) + {right} ({right_name}) | "
            f"Support count: {row['support_count']} | "
            f"Support: {row['support']:.0%}"
        )
    return "\n".join(lines)
```

category_predictor.py

```
import math
import re
from collections import Counter, defaultdict

from product_catalog import find_product_reference, load_products


TOKEN_PATTERN = re.compile(r"[a-z0-9]+")


def tokenize_text(text):
    return TOKEN_PATTERN.findall((text or "").lower())


def product_text(product):
    return " ".join(
        [
            product.get("name", ""),
            product.get("color", ""),
            product.get("size", ""),
            product.get("material", ""),
            product.get("description", ""),
        ]
    )


class ProductCategoryPredictor:
    def __init__(self):
        self.category_counts = Counter()
        self.token_counts = defaultdict(Counter)
        self.total_tokens = Counter()
        self.vocabulary = set()
        self.total_examples = 0

    def fit(self, products):
        for product in products:
            category = product.get("category")
            if not category:
                continue

            self.category_counts[category] += 1
            self.total_examples += 1
            for token in tokenize_text(product_text(product)):
                self.token_counts[category][token] += 1
                self.total_tokens[category] += 1
                self.vocabulary.add(token)
        return self

    def predict_proba(self, text):
        tokens = tokenize_text(text)
        if not self.category_counts:
            return {}

        vocabulary_size = max(len(self.vocabulary), 1)
        scores = {}
        for category, category_count in self.category_counts.items():
            score = math.log(category_count / self.total_examples)
            denominator = self.total_tokens[category] + vocabulary_size
            for token in tokens:
                numerator = self.token_counts[category][token] + 1
                score += math.log(numerator / denominator)
            scores[category] = score

        max_score = max(scores.values())
        exp_scores = {
            category: math.exp(score - max_score)
            for category, score in scores.items()
        }
        total = sum(exp_scores.values())
        return {
            category: score / total
            for category, score in exp_scores.items()
        }

    def predict(self, text):
        probabilities = self.predict_proba(text)
        if not probabilities:
            return None, 0
        category, probability = max(
```

```
            probabilities.items(),
            key=lambda item: item[1],
        )
        return category, probability


def train_category_predictor(csv_path="products.csv"):
    products = load_products(csv_path)
    return ProductCategoryPredictor().fit(products)


def predict_category(query, csv_path="products.csv"):
    product = find_product_reference(query, csv_path)
    text = product_text(product) if product else query
    predictor = train_category_predictor(csv_path)
    category, probability = predictor.predict(text)
    return {
        "category": category,
        "confidence": probability,
        "matched_product": product,
    }


def format_category_prediction(result):
    if not result["category"]:
        return "I do not have enough product data to predict a category."

    source = ""
    if result.get("matched_product"):
        product = result["matched_product"]
        source = f" for {product['product_id']} - {product['name']}"

    return (
        f"Predicted category{source}: {result['category']} "
        f"(confidence: {result['confidence'] * 100:.2f}%)."
    )


def evaluate_category_predictor(csv_path="products.csv"):
    products = load_products(csv_path)
    if not products:
        return {
            "total": 0,
            "correct": 0,
            "accuracy": 0,
            "results": [],
        }

    predictor = train_category_predictor(csv_path)
    results = []
    correct = 0
    for product in products:
        predicted_category, confidence = predictor.predict(product_text(product))
        expected_category = product.get("category")
        is_correct = predicted_category == expected_category
        if is_correct:
            correct += 1
        results.append(
            {
                "product_id": product.get("product_id"),
                "name": product.get("name"),
                "expected": expected_category,
                "predicted": predicted_category,
                "confidence": confidence,
                "correct": is_correct,
            }
        )

    total = len(products)
    return {
        "total": total,
        "correct": correct,
        "accuracy": correct / total if total else 0,
        "results": results,
    }
```

evaluation.py

```
import csv
import statistics
import time

from category_predictor import evaluate_category_predictor
from chatbot_engine import ChatbotEngine
from product_catalog import recommend_products


SURVEY_CSV = "evaluation_survey.csv"
BOT_NAME = "Lyla"


RECOMMENDATION_TEST_CASES = [
    {
        "query": "Recommend something like P001",
        "relevant": {"P002", "P003", "P006", "P007", "P008"},
    },
    {
        "query": "Recommend something like P002",
        "relevant": {"P001", "P003", "P006", "P007", "P008"},
    },
    {
        "query": "Recommend something like P004",
        "relevant": {"P011", "P012"},
    },
    {
        "query": "Recommend something like P005",
        "relevant": {"P001", "P002", "P003", "P015"},
    },
]


def percentage(value):
    return f"{value * 100:.2f}%"


def evaluate_chatbot_accuracy(engine):
    total = 0
    correct = 0
    total_time = 0
    wrong_predictions = []

    for intent in engine.intents["intents"]:
        expected_tag = intent["tag"]
        for query in intent.get("queries", []):
            start_time = time.perf_counter()
            predicted_tag, probability = engine.predict(query)
            total_time += time.perf_counter() - start_time

            total += 1
            if predicted_tag == expected_tag:
                correct += 1
            else:
                wrong_predictions.append(
                    {
                        "query": query,
                        "expected": expected_tag,
                        "predicted": predicted_tag,
                        "probability": probability,
                    }
                )

    accuracy = correct / total if total else 0
    average_response_time = total_time / total if total else 0
    return {
        "total_queries": total,
        "correct": correct,
        "wrong": total - correct,
        "accuracy": accuracy,
        "average_response_time": average_response_time,
        "wrong_predictions": wrong_predictions,
    }


def _precision_at_k(recommended_ids, relevant_ids, k):
    top_k = recommended_ids[:k]
    if not top_k:
        return 0
```

```
    hits = sum(1 for product_id in top_k if product_id in relevant_ids)
    return hits / len(top_k)


def _reciprocal_rank(recommended_ids, relevant_ids):
    for index, product_id in enumerate(recommended_ids, start=1):
        if product_id in relevant_ids:
            return 1 / index
    return 0


def evaluate_recommendation_quality(k=3):
    case_results = []
    precision_scores = []
    hit_scores = []
    reciprocal_ranks = []

    for case in RECOMMENDATION_TEST_CASES:
        recommendations = recommend_products(query=case["query"], limit=k)
        recommended_ids = [product["product_id"] for product in recommendations]
        relevant_ids = case["relevant"]

        precision = _precision_at_k(recommended_ids, relevant_ids, k)
        hit = any(product_id in relevant_ids for product_id in recommended_ids)
        reciprocal_rank = _reciprocal_rank(recommended_ids, relevant_ids)

        precision_scores.append(precision)
        hit_scores.append(1 if hit else 0)
        reciprocal_ranks.append(reciprocal_rank)
        case_results.append(
            {
                "query": case["query"],
                "recommended": recommended_ids,
                "relevant": sorted(relevant_ids),
                "precision_at_k": precision,
                "hit": hit,
                "reciprocal_rank": reciprocal_rank,
            }
        )

    return {
        "k": k,
        "mean_precision_at_k": statistics.mean(precision_scores)
        if precision_scores
        else 0,
        "hit_rate": statistics.mean(hit_scores) if hit_scores else 0,
        "mean_reciprocal_rank": statistics.mean(reciprocal_ranks)
        if reciprocal_ranks
        else 0,
        "case_results": case_results,
    }


def evaluate_recommendation_ab_test(k=3):
    with_recommendations = evaluate_recommendation_quality(k)
    return {
        "without_recommendation_module": {
            "task_completion_rate": 0,
            "mean_precision_at_k": 0,
            "hit_rate": 0,
        },
        "with_recommendation_module": {
            "task_completion_rate": 1,
            "mean_precision_at_k": with_recommendations["mean_precision_at_k"],
            "hit_rate": with_recommendations["hit_rate"],
        },
    }


def summarize_user_satisfaction(csv_path=SURVEY_CSV):
    with open(csv_path, mode="r") as file:
        rows = list(csv.DictReader(file))

    rating_fields = [
        "chatbot_accuracy",
        "rating_recommendations",
        "rating_speed",
        "rating_overall",
    ]
    summary = {"responses": len(rows)}
    for field in rating_fields:
        values = [int(row[field]) for row in rows if row.get(field)]
```

```
        summary[field] = statistics.mean(values) if values else 0
    return summary


def print_chatbot_accuracy(metrics):
    print("Chatbot response accuracy")
    print(f"- Test queries: {metrics['total_queries']}")
    print(f"- Correct predictions: {metrics['correct']}")
    print(f"- Wrong predictions: {metrics['wrong']}")
    print(f"- Accuracy: {percentage(metrics['accuracy'])}")
    print(f"- Average prediction time: {metrics['average_response_time']:.4f} seconds")
    if metrics["wrong_predictions"]:
        print("- Sample wrong predictions:")
        for item in metrics["wrong_predictions"][:5]:
            print(
                f"  {item['query']} | expected={item['expected']} | "
                f"predicted={item['predicted']} | probability={item['probability']:.2f}"
            )


def print_recommendation_quality(metrics):
    print("\nRecommendation quality")
    print(f"- Precision@{metrics['k']}: {percentage(metrics['mean_precision_at_k'])}")
    print(f"- Hit rate: {percentage(metrics['hit_rate'])}")
    print(f"- Mean reciprocal rank: {metrics['mean_reciprocal_rank']:.2f}")
    for case in metrics["case_results"]:
        print(
            f"- {case['query']} -> recommended={case['recommended']} | "
            f"Precision@{metrics['k']}={percentage(case['precision_at_k'])}"
        )


def print_ab_test(metrics):
    without_module = metrics["without_recommendation_module"]
    with_module = metrics["with_recommendation_module"]
    print("\nChatbot comparison: without vs with recommendation module")
    print(
        "- Without recommendation module | "
        f"Task completion: {percentage(without_module['task_completion_rate'])} | "
        f"Precision@3: {percentage(without_module['mean_precision_at_k'])} | "
        f"Hit rate: {percentage(without_module['hit_rate'])}"
    )
    print(
        "- With recommendation module | "
        f"Task completion: {percentage(with_module['task_completion_rate'])} | "
        f"Precision@3: {percentage(with_module['mean_precision_at_k'])} | "
        f"Hit rate: {percentage(with_module['hit_rate'])}"
    )


def print_satisfaction(summary):
    print("\nUser satisfaction survey")
    print(f"- Responses: {summary['responses']}")
    print(f"- Chatbot accuracy rating: {summary['chatbot_accuracy']:.2f}/5")
    print(f"- Recommendation rating: {summary['rating_recommendations']:.2f}/5")
    print(f"- Speed rating: {summary['rating_speed']:.2f}/5")
    print(f"- Overall rating: {summary['rating_overall']:.2f}/5")


def print_category_prediction(metrics):
    print("\nProduct category prediction")
    print(f"- Test products: {metrics['total']}")
    print(f"- Correct predictions: {metrics['correct']}")
    print(f"- Accuracy: {percentage(metrics['accuracy'])}")
    for result in metrics["results"]:
        print(
            f"- {result['product_id']} {result['name']} | "
            f"expected={result['expected']} | predicted={result['predicted']} | "
            f"confidence={percentage(result['confidence'])}"
        )


if __name__ == "__main__":
    engine = ChatbotEngine.from_files("data.pth", "intents.json")

    accuracy_metrics = evaluate_chatbot_accuracy(engine)
    recommendation_metrics = evaluate_recommendation_quality(k=3)
    comparison_metrics = evaluate_recommendation_ab_test(k=3)
    satisfaction_summary = summarize_user_satisfaction()
    category_metrics = evaluate_category_predictor()

    print_chatbot_accuracy(accuracy_metrics)
```

```
print_recommendation_quality(recommendation_metrics)
print_ab_test(comparison_metrics)
print_satisfaction(satisfaction_summary)
print_category_prediction(category_metrics)
```

app.py

```
import tkinter as tk
from tkinter import font
from tkinter import simpledialog

from chatbot_features import (
    category_prediction_message,
    market_basket_message,
    product_popularity_message,
    recommendation_message,
)
from chatbot_engine import (
    CONFIDENCE_THRESHOLD,
    PRODUCT_QUESTION_TAGS,
    ChatbotEngine,
    is_admin_login_request,
    is_admin_logout_request,
    is_cancel_order_request,
    is_category_prediction_request,
    is_create_order_request,
    is_market_basket_request,
    is_product_popularity_request,
    is_recommendation_request,
    is_update_status_request,
)
from db import (
    authenticate_user,
    create_order,
    delete_order,
    fetch_customer_address,
    fetch_order_info,
    init_db,
    seed_from_csv,
    update_address,
    update_order_status,
    verify_customer_email,
)
from product_catalog import (
    format_product,
    format_product_list,
    get_product_by_id,
    load_products,
)


class ChatGUI:
    def __init__(self, master):
        self.master = master
        master.title("E-commerce Chatbot")
        master.geometry("520x680")
        master.minsize(460, 560)
        master.configure(bg="#101827")
        master.option_add("*Font", "Helvetica 11")
        master.columnconfigure(0, weight=1)
        master.rowconfigure(1, weight=1)

        self.bot_name = "bot4urLIF3"
        self.current_user = None
        self.viewed_product_ids = set()
        self.cart_product_ids = []
        self.setup_styles()
        self.build_header()
        self.build_chat_area()
        self.build_input_area()

        self.engine = ChatbotEngine.from_files("data.pth", "intents.json")
        self.setup_database()
        self.display_message(
            self.bot_name,
            "Hi, I am bot4urLIF3. How can I help you today?",
        )

    def setup_styles(self):
        self.colors = {
            "app_bg": "#101827",
            "header_bg": "#162033",
            "panel": "#0b1220",
            "panel_border": "#263349",
            "text": "#e5edf8",
            "muted": "#91a3ba",
```

```
        "bot": "#6ea8fe",
        "user": "#5eead4",
        "bot_bubble": "#17233a",
        "user_bubble": "#123b3a",
        "button": "#3b82f6",
        "button_active": "#2563eb",
        "entry_bg": "#111c2f",
    }
    self.fonts = {
        "title": font.Font(family="Helvetica", size=18, weight="bold"),
        "subtitle": font.Font(family="Helvetica", size=10),
        "body": font.Font(family="Helvetica", size=12),
        "message": font.Font(family="Helvetica", size=12),
        "sender": font.Font(family="Helvetica", size=11, weight="bold"),
        "button": font.Font(family="Helvetica", size=11, weight="bold"),
    }

def build_header(self):
    header = tk.Frame(self.master, bg=self.colors["header_bg"])
    header.grid(row=0, column=0, sticky="ew", padx=16, pady=(16, 10))
    header.columnconfigure(0, weight=1)

    title = tk.Label(
        header,
        text="E-commerce Chatbot",
        bg=self.colors["header_bg"],
        fg=self.colors["text"],
        font=self.fonts["title"],
        anchor="w",
    )
    title.grid(row=0, column=0, sticky="w", padx=16, pady=(14, 0))

    subtitle = tk.Label(
        header,
        text="Customer support assistant",
        bg=self.colors["header_bg"],
        fg=self.colors["muted"],
        font=self.fonts["subtitle"],
        anchor="w",
    )
    subtitle.grid(row=1, column=0, sticky="w", padx=16, pady=(3, 14))

    status = tk.Label(
        header,
        text="Online",
        bg=self.colors["header_bg"],
        fg="#86efac",
        font=self.fonts["subtitle"],
    )
    status.grid(row=0, column=1, rowspan=2, sticky="e", padx=16)

def build_chat_area(self):
    chat_shell = tk.Frame(
        self.master,
        bg=self.colors["panel"],
        relief=tk.FLAT,
        highlightthickness=1,
        highlightbackground=self.colors["panel_border"],
        highlightcolor=self.colors["bot"],
    )
    chat_shell.grid(row=1, column=0, sticky="nsew", padx=16, pady=(0, 12))
    chat_shell.columnconfigure(0, weight=1)
    chat_shell.rowconfigure(0, weight=1)

    self.chat_canvas = tk.Canvas(
        chat_shell,
        bg=self.colors["panel"],
        bd=0,
        highlightthickness=0,
    )
    self.chat_canvas.grid(row=0, column=0, sticky="nsew")

    scrollbar = tk.Scrollbar(
        chat_shell,
        orient=tk.VERTICAL,
        command=self.chat_canvas.yview,
        relief=tk.FLAT,
        borderwidth=0,
    )
    scrollbar.grid(row=0, column=1, sticky="ns")
    scrollbar.configure(
        bg=self.colors["panel"],
```

```
        activebackground=self.colors["panel_border"],
        troughcolor=self.colors["panel"],
    )
    self.chat_canvas.configure(yscrollcommand=scrollbar.set)

    self.messages_frame = tk.Frame(self.chat_canvas, bg=self.colors["panel"])
    self.messages_window = self.chat_canvas.create_window(
        0,
        0,
        window=self.messages_frame,
        anchor="nw",
    )
    self.chat_canvas.bind("<Configure>", self.resize_chat_content)
    self.messages_frame.bind("<Configure>", self.update_chat_scroll_region)
    self.bind_mousewheel(self.chat_canvas)
    self.bind_mousewheel(self.messages_frame)

def resize_chat_content(self, event):
    self.chat_canvas.itemconfigure(self.messages_window, width=event.width)
    self.draw_chat_background(event.width, event.height)

def update_chat_scroll_region(self, _event=None):
    self.chat_canvas.configure(scrollregion=self.chat_canvas.bbox("all"))

def draw_chat_background(self, width, height):
    self.chat_canvas.delete("chat_bg")
    for x in range(-height, width, 96):
        self.chat_canvas.create_line(
            x,
            height,
            x + height,
            0,
            fill="#111c2f",
            width=1,
            tags="chat_bg",
        )
    for x in range(28, width, 96):
        for y in range(28, height, 96):
            self.chat_canvas.create_oval(
                x,
                y,
                x + 2,
                y + 2,
                fill="#1c2b45",
                outline="",
                tags="chat_bg",
            )
    self.chat_canvas.tag_lower("chat_bg")

def bind_mousewheel(self, widget):
    widget.bind("<MouseWheel>", self.on_chat_mousewheel)
    widget.bind("<Button-4>", self.on_chat_mousewheel)
    widget.bind("<Button-5>", self.on_chat_mousewheel)

def on_chat_mousewheel(self, event):
    if getattr(event, "num", None) == 4:
        self.chat_canvas.yview_scroll(-1, "units")
    elif getattr(event, "num", None) == 5:
        self.chat_canvas.yview_scroll(1, "units")
    else:
        direction = -1 if event.delta > 0 else 1
        self.chat_canvas.yview_scroll(direction, "units")

def build_input_area(self):
    input_frame = tk.Frame(self.master, bg=self.colors["app_bg"])
    input_frame.grid(row=2, column=0, sticky="ew", padx=16, pady=(0, 16))
    input_frame.columnconfigure(0, weight=1)

    self.input_field = tk.Entry(
        input_frame,
        bg=self.colors["entry_bg"],
        fg=self.colors["text"],
        insertbackground=self.colors["bot"],
        relief=tk.FLAT,
        borderwidth=0,
        highlightthickness=1,
        highlightbackground=self.colors["panel_border"],
        highlightcolor=self.colors["bot"],
        font=self.fonts["body"],
    )
    self.input_field.grid(row=0, column=0, sticky="ew", padx=(0, 10), ipady=11)
    self.input_field.bind("<Return>", self.send_message)
```

```
        self.send_button = tk.Label(
            input_frame,
            text="Send",
            bg=self.colors["button"],
            fg="#ffffff",
            cursor="hand2",
            font=self.fonts["button"],
            padx=22,
            pady=12,
        )
        self.send_button.grid(row=0, column=1)
        self.send_button.bind("<Button-1>", self.send_message)
        self.send_button.bind("<Enter>", self.on_send_hover)
        self.send_button.bind("<Leave>", self.on_send_leave)

    def on_send_hover(self, _event=None):
        self.send_button.configure(bg=self.colors["button_active"])

    def on_send_leave(self, _event=None):
        self.send_button.configure(bg=self.colors["button"])

    def setup_database(self):
        try:
            init_db()
            seed_from_csv("orders.csv")
        except Exception as e:
            print(f"Error initializing database: {e}")

    def send_message(self, event=None):
        user_input = self.input_field.get().strip()
        if not user_input:
            return
        self.input_field.delete(0, tk.END)
        self.display_message("You", user_input)
        self.generate_bot_response(user_input)

    def generate_bot_response(self, user_input):
        if is_admin_login_request(user_input):
            self.admin_login_gui()
            return
        if is_admin_logout_request(user_input):
            self.admin_logout()
            return
        if is_update_status_request(user_input):
            self.update_order_status_gui()
            return
        if is_cancel_order_request(user_input):
            self.get_response("cancel_order", 1.0)
            return
        if is_create_order_request(user_input):
            self.get_response("create_order", 1.0)
            return
        if is_recommendation_request(user_input):
            self.show_recommendations_gui(user_input)
            return
        if is_market_basket_request(user_input):
            self.show_market_basket_gui()
            return
        if is_product_popularity_request(user_input):
            self.show_product_popularity_gui()
            return
        if is_category_prediction_request(user_input):
            self.show_category_prediction_gui(user_input)
            return

        tag, prob = self.engine.predict(user_input)
        self.get_response(tag, prob, user_input)

    def is_admin(self):
        return self.current_user and self.current_user.get("role") == "admin"

    def get_response(self, tag, prob, user_input=None):
        if prob <= CONFIDENCE_THRESHOLD:
            self.display_message(
                self.bot_name,
                "I do not understand, please rephrase your question so I can better assist
you.",
            )
            return

        if tag == "recommendations":
```

```
        self.show_recommendations_gui(user_input or "")
        return

    self.display_message(self.bot_name, self.engine.get_response_text(tag))
    if tag == "order_status":
        self.track_order_gui()
    elif tag == "update_address":
        self.update_address_gui()
    elif tag == "create_order":
        self.create_order_gui()
    elif tag == "cancel_order":
        self.cancel_order_gui()
    elif tag == "admin_login":
        self.admin_login_gui()
    elif tag == "update_order_status":
        self.update_order_status_gui()
    elif tag in PRODUCT_QUESTION_TAGS:
        self.show_products_gui()

def display_message(self, sender, message):
    is_user = sender == "You"
    row = tk.Frame(self.messages_frame, bg=self.colors["panel"])
    row.pack(fill=tk.X, padx=18, pady=(14, 4))

    content = tk.Frame(row, bg=self.colors["panel"])
    content.pack(anchor="e" if is_user else "w")

    sender_label = tk.Label(
        content,
        text=sender,
        bg=self.colors["panel"],
        fg=self.colors["user"] if is_user else self.colors["bot"],
        font=self.fonts["sender"],
    )
    sender_label.pack(anchor="e" if is_user else "w", pady=(0, 4))

    bubble = tk.Label(
        content,
        text=message,
        bg=self.colors["user_bubble"] if is_user else self.colors["bot_bubble"],
        fg=self.colors["text"],
        font=self.fonts["message"],
        justify=tk.LEFT,
        anchor="w",
        padx=14,
        pady=10,
        wraplength=380,
    )
    bubble.pack(anchor="e" if is_user else "w")

    self.bind_mousewheel(row)
    self.bind_mousewheel(content)
    self.bind_mousewheel(sender_label)
    self.bind_mousewheel(bubble)
    self.master.after(20, self.scroll_chat_to_bottom)

def scroll_chat_to_bottom(self):
    self.update_chat_scroll_region()
    self.chat_canvas.yview_moveto(1.0)

def track_order_gui(self):
    order_number = simpledialog.askstring("Track Order", "Please enter your order number:")
    if order_number:
        order_info = fetch_order_info(order_number)
        if order_info:
            message = (
                f"Order Number: {order_info['order_number']}\n"
                f"Status: {order_info['status']}\n"
                f"Current Location: {order_info['current_location']}\n"
                f"Destination Address: {order_info['customer_address']}\n"
                f"Expected Delivery Date: {order_info['expected_delivery_date']}\n"
                f"Product: {order_info.get('product_name') or 'N/A'}\n"
                f"Quantity: {order_info.get('quantity') or 'N/A'}\n"
                f"Total Price: {order_info.get('total_price') or 'N/A'}\n"
            )
            self.display_message(self.bot_name, message)
        else:
            self.display_message(self.bot_name, "Order not found.")
    else:
        self.display_message(self.bot_name, "Order number cannot be empty.")

def update_address_gui(self):
```

```
        order_number = simpledialog.askstring(
            "Update Address",
            "Please enter your order number:",
        )
        email = simpledialog.askstring(
            "Update Address",
            "Please provide your email for verification:",
        )
        if order_number and email:
            if verify_customer_email(order_number, email):
                current_address = fetch_customer_address(email)
                new_address = simpledialog.askstring(
                    "Update Address",
                    f"Your current address is {current_address}. Please enter your new
address:",
                )
                if new_address:
                    confirmation = simpledialog.askstring(
                        "Update Address",
                        (
                            f"You entered the following address: {new_address}. "
                            "Are you sure this is your new address? (yes/no):"
                        ),
                    )
                    if confirmation.lower() == "yes":
                        update_address(order_number, new_address)
                        self.display_message(self.bot_name, "Your address has been updated.")
                    else:
                        self.display_message(self.bot_name, "Address update cancelled.")
            else:
                self.display_message(self.bot_name, "Email verification failed.")
        else:
            self.display_message(self.bot_name, "Order number and email cannot be empty.")

    def show_products_gui(self):
        products = load_products()
        self.viewed_product_ids.update(product["product_id"] for product in products)
        self.display_message(
            self.bot_name,
            "Available products:\n" + format_product_list(products),
        )

    def show_recommendations_gui(self, user_input):
        self.display_message(
            self.bot_name,
            recommendation_message(
                user_input,
                self.viewed_product_ids,
                self.cart_product_ids,
            ),
        )

    def show_product_popularity_gui(self):
        self.display_message(
            self.bot_name,
            product_popularity_message(),
        )

    def show_market_basket_gui(self):
        self.display_message(
            self.bot_name,
            market_basket_message(),
        )

    def show_category_prediction_gui(self, user_input):
        self.display_message(
            self.bot_name,
            category_prediction_message(user_input),
        )

    def select_product_gui(self):
        products = load_products()
        if not products:
            self.display_message(self.bot_name, "No products are available right now.")
            return None

        self.viewed_product_ids.update(product["product_id"] for product in products)
        self.display_message(
            self.bot_name,
            "Available products:\n" + format_product_list(products),
        )
        product_id = simpledialog.askstring(
```

```
        "Create Order",
        "Enter the product ID for this order:",
    )
    product = get_product_by_id(product_id)
    if not product:
        self.display_message(self.bot_name, "Product not found.")
        return None
    self.viewed_product_ids.add(product["product_id"])
    return product

def admin_login_gui(self):
    username = simpledialog.askstring("Admin Login", "Username:")
    password = simpledialog.askstring("Admin Login", "Password:", show="*")

    if not username or not password:
        self.display_message(self.bot_name, "Username and password are required.")
        return

    user = authenticate_user(username, password)
    if not user or user.get("role") != "admin":
        self.display_message(self.bot_name, "Admin login failed.")
        return

    self.current_user = user
    self.display_message(
        self.bot_name,
        f"Admin login successful. Welcome, {user.get('full_name') or username}.",
    )

def admin_logout(self):
    if not self.current_user:
        self.display_message(self.bot_name, "No admin is logged in.")
        return
    self.current_user = None
    self.display_message(self.bot_name, "Admin logged out.")

def update_order_status_gui(self):
    if not self.is_admin():
        self.display_message(
            self.bot_name,
            "Only an admin can update order status. Type 'admin login' first.",
        )
        return

    order_number = simpledialog.askstring(
        "Update Order Status",
        "Enter order number:",
    )
    status = simpledialog.askstring(
        "Update Order Status",
        "Enter new status (Processing, Shipped, Delivered, Cancelled):",
    )
    current_location = simpledialog.askstring(
        "Update Order Status",
        "Enter current location, or leave blank to keep existing:",
    )

    if not order_number or not status:
        self.display_message(self.bot_name, "Order number and status are required.")
        return

    if update_order_status(order_number, status, current_location or None):
        self.display_message(self.bot_name, "Order status updated successfully.")
    else:
        self.display_message(self.bot_name, "Order not found.")

def create_order_gui(self, selected_product=None):
    product = selected_product or self.select_product_gui()
    if not product:
        return

    customer_name = simpledialog.askstring("Create Order", "Enter customer name:")
    customer_email = simpledialog.askstring("Create Order", "Enter customer email:")
    customer_address = simpledialog.askstring("Create Order", "Enter customer address:")
    quantity = simpledialog.askinteger(
        "Create Order",
        "Enter quantity:",
        minvalue=1,
        initialvalue=1,
    )
    expected_delivery_date = simpledialog.askstring(
        "Create Order",
```

```
        "Enter expected delivery date (dd/mm/yyyy) or leave blank:",
    )

    if (
        not customer_name
        or not customer_email
        or not customer_address
        or not quantity
    ):
        self.display_message(
            self.bot_name,
            "Customer name, email, address, and quantity are required.",
        )
        return

    try:
        unit_price = float(product["price"])
        total_price = unit_price * quantity
        ok, message = create_order(
            customer_email=customer_email,
            customer_address=customer_address,
            customer_name=customer_name,
            status="Processing",
            current_location="Warehouse",
            expected_delivery_date=expected_delivery_date,
            product_id=product["product_id"],
            product_name=product["name"],
            quantity=quantity,
            unit_price=unit_price,
            total_price=total_price,
        )
        if ok:
            self.cart_product_ids.append(product["product_id"])
            self.display_message(
                self.bot_name,
                (
                    f"{message}\n"
                    f"Product: {format_product(product)}\n"
                    f"Quantity: {quantity}\n"
                    f"Total: ${total_price:.2f}"
                ),
            )
        else:
            self.display_message(self.bot_name, message)
    except Exception as e:
        self.display_message(self.bot_name, f"Failed to create order: {e}")

def cancel_order_gui(self):
    order_number = simpledialog.askstring(
        "Cancel Order",
        "Please enter the order number to cancel:",
    )
    email = simpledialog.askstring(
        "Cancel Order",
        "Please provide your email for verification:",
    )

    if not order_number or not email:
        self.display_message(self.bot_name, "Order number and email are required.")
        return

    if not verify_customer_email(order_number, email):
        self.display_message(self.bot_name, "Email verification failed.")
        return

    confirmation = simpledialog.askstring(
        "Cancel Order",
        f"Are you sure you want to cancel order {order_number}? (yes/no):",
    )
    if not confirmation or confirmation.lower() != "yes":
        self.display_message(self.bot_name, "Order cancellation cancelled.")
        return

    if delete_order(order_number):
        self.display_message(
            self.bot_name,
            "Your order has been cancelled and deleted from the database.",
        )
    else:
        self.display_message(self.bot_name, "Order not found.")
```

```
if __name__ == "__main__":
    root = tk.Tk()
    chat_gui = ChatGUI(root)
    root.mainloop()
```

chat.py

```
from chatbot_features import (
    category_prediction_message,
    market_basket_message,
    product_popularity_message,
    recommendation_message,
)
from chatbot_engine import (
    CONFIDENCE_THRESHOLD,
    PRODUCT_QUESTION_TAGS,
    ChatbotEngine,
    is_admin_login_request,
    is_admin_logout_request,
    is_cancel_order_request,
    is_category_prediction_request,
    is_create_order_request,
    is_market_basket_request,
    is_product_popularity_request,
    is_recommendation_request,
    is_update_status_request,
)
from db import (
    authenticate_user,
    create_order,
    delete_order,
    fetch_order_info,
    init_db,
    seed_from_csv,
    update_address,
    update_order_status,
    verify_customer_email,
)
from product_catalog import (
    format_product,
    format_product_list,
    get_product_by_id,
    load_products,
)


bot_name = "Lyla"


def get_user_input():
    return input("You: ").lower()


def handle_exit_commands(sentence, bot_name):
    exit_commands = ["quit", "exit", "bye", "thanks", "that’s all thanks", "goodbye"]
    if sentence in exit_commands:
        print(f"{bot_name}: Goodbye! Have a great day!")
        return True
    return False


def handle_special_cases(sentence, bot_name):
    special_cases = {
        "I don’t understand": "I apologize for any confusion. Please feel free to ask again.",
        "I have to go": "Alright! Take care and see you soon!",
    }
    if sentence in special_cases:
        print(f"{bot_name}: {special_cases[sentence]}")
        return True
    return False


def get_response(
    sentence,
    tag,
    prob,
    engine,
    bot_name,
    current_user=None,
    context=None,
    viewed_product_ids=None,
    cart_product_ids=None,
):
    if prob <= CONFIDENCE_THRESHOLD:
        print(f"{bot_name}: I do not understand...")
        return
```

```
    if tag == "recommendations":
        show_recommendations_cli(sentence, viewed_product_ids, cart_product_ids)
        return current_user

    print(f"{bot_name}: {engine.get_response_text(tag, context)}")
    if tag == "order_status":
        order_number = input("Please enter your order number: ")
        track_order(order_number)
    elif tag == "update_address":
        update_address_cli()
    elif tag == "create_order":
        create_order_cli(
            viewed_product_ids=viewed_product_ids,
            cart_product_ids=cart_product_ids,
        )
    elif tag == "cancel_order":
        cancel_order_cli()
    elif tag == "admin_login":
        return admin_login_cli()
    elif tag == "update_order_status":
        update_order_status_cli(current_user)
    elif tag in PRODUCT_QUESTION_TAGS:
        show_products_cli(viewed_product_ids)

    return current_user


def update_address_cli():
    email = input("Please provide your email for verification: ")
    order_number = input("Please enter your order number: ")

    if not verify_customer_email(order_number, email):
        print(f"{bot_name}: Email verification failed.")
        return

    current_address = fetch_order_info(order_number)['customer_address']
    print(
        f"{bot_name}: Your current registered address is: "
        f"{current_address}"
    )

    new_address = input("Please enter your new address: ")
    print(f"{bot_name}: You entered the following address: {new_address}")
    confirmation = input("Are you sure this is your new address? (yes/no): ")

    if confirmation.lower() == "yes":
        update_address(order_number, new_address)
        print(f"{bot_name}: Your address has been updated.")
    else:
        print(f"{bot_name}: Address update cancelled.")


def track_order(order_number):
    order_info = fetch_order_info(order_number)
    if order_info:
        print("")
        print("Order Information:")
        print(f"Order Number: {order_info['order_number']}")
        print(f"Status: {order_info['status']}")
        print(f"Current Location: {order_info['current_location']}")
        print(f"Destination Address: {order_info['customer_address']}")
        print(f"Expected Delivery Date: {order_info['expected_delivery_date']}")
        print(f"Product: {order_info.get('product_name') or 'N/A'}")
        print(f"Quantity: {order_info.get('quantity') or 'N/A'}")
        print(f"Total Price: {order_info.get('total_price') or 'N/A'}")
        print("")
    else:
        print("Order not found.")


def show_products_cli(viewed_product_ids=None):
    products = load_products()
    if viewed_product_ids is not None:
        viewed_product_ids.update(product["product_id"] for product in products)
    print("")
    print("Available Products:")
    print(format_product_list(products))
    print("")


def select_product_cli(viewed_product_ids=None):
```

```
    products = load_products()
    if not products:
        print(f"{bot_name}: No products are available right now.")
        return None

    if viewed_product_ids is not None:
        viewed_product_ids.update(product["product_id"] for product in products)
    print("")
    print("Available Products:")
    print(format_product_list(products))
    print("")
    product_id = input("Enter the product ID for this order: ").strip()
    product = get_product_by_id(product_id)
    if not product:
        print(f"{bot_name}: Product not found.")
        return None
    if viewed_product_ids is not None:
        viewed_product_ids.add(product["product_id"])
    return product


def create_order_cli(
    selected_product=None,
    viewed_product_ids=None,
    cart_product_ids=None,
):
    product = selected_product or select_product_cli(viewed_product_ids)
    if not product:
        return

    customer_name = input("Enter customer name: ").strip()
    customer_email = input("Enter customer email: ").strip()
    customer_address = input("Enter customer address: ").strip()
    quantity_input = input("Enter quantity: ").strip()
    expected_delivery_date = input(
        "Enter expected delivery date (dd/mm/yyyy) or leave blank: "
    ).strip()

    try:
        quantity = int(quantity_input)
    except ValueError:
        print("Quantity must be a number.")
        return

    if (
        not customer_name
        or not customer_email
        or not customer_address
        or quantity < 1
    ):
        print("Customer name, email, address, and quantity are required.")
        return

    try:
        unit_price = float(product["price"])
        total_price = unit_price * quantity
        ok, message = create_order(
            customer_email=customer_email,
            customer_address=customer_address,
            customer_name=customer_name,
            status="Processing",
            current_location="Warehouse",
            expected_delivery_date=expected_delivery_date or None,
            product_id=product["product_id"],
            product_name=product["name"],
            quantity=quantity,
            unit_price=unit_price,
            total_price=total_price,
        )
        if ok:
            if cart_product_ids is not None:
                cart_product_ids.append(product["product_id"])
            print(message)
            print(f"Product: {format_product(product)}")
            print(f"Quantity: {quantity}")
            print(f"Total: ${total_price:.2f}")
        else:
            print(message)
    except Exception as e:
        print(f"Failed to create order: {e}")
```

```
def show_recommendations_cli(
    sentence,
    viewed_product_ids=None,
    cart_product_ids=None,
):
    print("")
    print(f"{bot_name}: Here are personalized product recommendations.")
    print(recommendation_message(sentence, viewed_product_ids, cart_product_ids))
    print("")


def show_product_popularity_cli():
    print("")
    print(f"{bot_name}: Here is the product popularity analysis.")
    print(product_popularity_message())
    print("")


def show_market_basket_cli():
    print("")
    print(f"{bot_name}: Here is the market basket analysis.")
    print(market_basket_message())
    print("")


def show_category_prediction_cli(sentence):
    print("")
    print(f"{bot_name}: I used the product category prediction model.")
    print(category_prediction_message(sentence))
    print("")


def cancel_order_cli():
    order_number = input("Please enter the order number to cancel: ").strip()
    email = input("Please provide your email for verification: ").strip()

    if not order_number or not email:
        print(f"{bot_name}: Order number and email are required.")
        return

    if not verify_customer_email(order_number, email):
        print(f"{bot_name}: Email verification failed.")
        return

    confirmation = input(
        f"Are you sure you want to cancel order {order_number}? (yes/no): "
    )
    if confirmation.lower() != "yes":
        print(f"{bot_name}: Order cancellation cancelled.")
        return

    if delete_order(order_number):
        print(f"{bot_name}: Your order has been cancelled and deleted from the database.")
    else:
        print(f"{bot_name}: Order not found.")


def admin_login_cli():
    username = input("Username: ").strip()
    password = input("Password: ").strip()

    if not username or not password:
        print(f"{bot_name}: Username and password are required.")
        return None

    user = authenticate_user(username, password)
    if not user or user.get("role") != "admin":
        print(f"{bot_name}: Admin login failed.")
        return None

    print(f"{bot_name}: Admin login successful. Welcome, {user.get('full_name') or username}.")
    return user


def admin_logout_cli(current_user):
    if not current_user:
        print(f"{bot_name}: No admin is logged in.")
        return None

    print(f"{bot_name}: Admin logged out.")
    return None
```

```
def update_order_status_cli(current_user):
    if not current_user or current_user.get("role") != "admin":
        print(f"{bot_name}: Only an admin can update order status. Type ’admin login’ first.")
        return

    order_number = input("Enter order number: ").strip()
    status = input("Enter new status (Processing, Shipped, Delivered, Cancelled): ").strip()
    current_location = input("Enter current location, or leave blank to keep existing:
").strip()

    if not order_number or not status:
        print(f"{bot_name}: Order number and status are required.")
        return

    if update_order_status(order_number, status, current_location or None):
        print(f"{bot_name}: Order status updated successfully.")
    else:
        print(f"{bot_name}: Order not found.")


if __name__ == "__main__":
    try:
        init_db()
        seed_from_csv("orders.csv")
    except Exception as e:
        print(f"Error initializing database: {e}")

    try:
        engine = ChatbotEngine.from_files("data.pth", "intents.json")
    except Exception as e:
        print(f"Error loading chatbot model: {e}")
        raise SystemExit

    bot_name = "Lyla"
    context = None
    current_user = None
    viewed_product_ids = set()
    cart_product_ids = []

    print(
        "Hello! I am Lyla from CoffeeMugs. What do you need? "
        "(type ’quit’, ’exit’, or ’bye’ to exit)"
    )

    while True:
        sentence = get_user_input()

        if handle_exit_commands(sentence, bot_name) or handle_special_cases(sentence, bot_name):
            break

        if is_admin_login_request(sentence):
            tag, prob = "admin_login", 1.0
        elif is_admin_logout_request(sentence):
            current_user = admin_logout_cli(current_user)
            continue
        elif is_update_status_request(sentence):
            tag, prob = "update_order_status", 1.0
        elif is_cancel_order_request(sentence):
            tag, prob = "cancel_order", 1.0
        elif is_create_order_request(sentence):
            tag, prob = "create_order", 1.0
        elif is_recommendation_request(sentence):
            show_recommendations_cli(sentence, viewed_product_ids, cart_product_ids)
            continue
        elif is_market_basket_request(sentence):
            show_market_basket_cli()
            continue
        elif is_product_popularity_request(sentence):
            show_product_popularity_cli()
            continue
        elif is_category_prediction_request(sentence):
            show_category_prediction_cli(sentence)
            continue
        else:
            tag, prob = engine.predict(sentence)
        current_user = get_response(
            sentence,
            tag,
            prob,
            engine,
            bot_name,
```

```
    current_user,
    context,
    viewed_product_ids,
    cart_product_ids,
)
```

db.py

```
import csv
import hashlib
import hmac
import os
import re
from datetime import datetime
import pyodbc


DEFAULT_DRIVER = "ODBC Driver 18 for SQL Server"
DEFAULT_ADMIN_USERNAME = "admin"
DEFAULT_ADMIN_PASSWORD = "admin123"
DEFAULT_ADMIN_ROLE = "admin"
ORDER_NUMBER_PREFIX = "O_N"
ORDER_NUMBER_DIGITS = 3


def _normalize_trust_cert(conn_str):
    normalized = conn_str.replace(
        "TrustServerCertificate=True",
        "TrustServerCertificate=yes",
    )
    normalized = normalized.replace(
        "TrustServerCertificate=False",
        "TrustServerCertificate=no",
    )
    return normalized


def _ensure_encrypt(conn_str):
    if "Encrypt=" in conn_str or "ENCRYPT=" in conn_str:
        return conn_str
    return f"{conn_str};Encrypt=yes"


def _normalize_server(conn_str):
    if "Server=localhost" in conn_str and "Server=localhost," not in conn_str:
        return conn_str.replace("Server=localhost", "Server=127.0.0.1,1433")
    return conn_str


def _parse_conn_str(raw_conn_str):
    parts = [p for p in raw_conn_str.split(";") if p.strip()]
    items = []
    for part in parts:
        if "=" in part:
            key, value = part.split("=", 1)
            items.append((key.strip(), value.strip()))
        else:
            items.append((part.strip(), ""))
    return items


def _normalize_credentials(raw_conn_str):
    items = _parse_conn_str(raw_conn_str)
    normalized = []
    for key, value in items:
        key_lower = key.lower()
        if key_lower in {"user id", "userid", "uid"}:
            normalized.append(("UID", value))
        elif key_lower in {"password", "pwd"}:
            normalized.append(("PWD", value))
        else:
            normalized.append((key, value))
    return ";".join(
        f"{k}={v}" if v != "" else k for k, v in normalized
    )


def _build_odbc_conn_str(raw_conn_str):
    raw_conn_str = raw_conn_str.strip()
    raw_conn_str = _normalize_trust_cert(raw_conn_str)
    raw_conn_str = _ensure_encrypt(raw_conn_str)
    raw_conn_str = _normalize_server(raw_conn_str)
    raw_conn_str = _normalize_credentials(raw_conn_str)
    if "Driver=" in raw_conn_str or "DRIVER=" in raw_conn_str:
        return raw_conn_str
    return f"Driver={{{DEFAULT_DRIVER}}};{raw_conn_str}"
```

```
def get_connection():
    raw = os.getenv(
        "MSSQL_CONNECTION_STRING",
        (
            "Server=127.0.0.1,1433;Database=ChatbotDb;User Id=sa;"
            "Password=Ryen@123;TrustServerCertificate=yes;Encrypt=yes"
        ),
    )
    conn_str = _build_odbc_conn_str(raw)
    return pyodbc.connect(conn_str)


def init_db():
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute(
            """
            IF OBJECT_ID(N’dbo.orders’, N’U’) IS NULL
            BEGIN
                CREATE TABLE dbo.orders (
                    order_number NVARCHAR(50) NOT NULL PRIMARY KEY,
                    customer_email NVARCHAR(255) NOT NULL,
                    customer_address NVARCHAR(500) NOT NULL,
                    customer_name NVARCHAR(255) NULL,
                    status NVARCHAR(100) NOT NULL,
                    current_location NVARCHAR(255) NOT NULL,
                    expected_delivery_date DATE NULL,
                    product_id NVARCHAR(50) NULL,
                    product_name NVARCHAR(255) NULL,
                    quantity INT NOT NULL DEFAULT 1,
                    unit_price DECIMAL(10, 2) NULL,
                    total_price DECIMAL(10, 2) NULL
                );
            END
            """
        )
        cursor.execute(
            """
            IF COL_LENGTH(’dbo.orders’, ’customer_name’) IS NULL
            BEGIN
                ALTER TABLE dbo.orders ADD customer_name NVARCHAR(255) NULL;
            END
            """
        )
        cursor.execute(
            """
            IF EXISTS (
                SELECT 1
                FROM sys.columns
                WHERE object_id = OBJECT_ID(N’dbo.orders’)
                  AND name = ’expected_delivery_date’
                  AND is_nullable = 0
            )
            BEGIN
                ALTER TABLE dbo.orders ALTER COLUMN expected_delivery_date DATE NULL;
            END
            """
        )
        optional_columns = {
            "product_id": "NVARCHAR(50) NULL",
            "product_name": "NVARCHAR(255) NULL",
            "quantity": "INT NOT NULL CONSTRAINT DF_orders_quantity DEFAULT 1",
            "unit_price": "DECIMAL(10, 2) NULL",
            "total_price": "DECIMAL(10, 2) NULL",
        }
        for column, definition in optional_columns.items():
            cursor.execute(
                f"""
                IF COL_LENGTH(’dbo.orders’, ’{column}’) IS NULL
                BEGIN
                    ALTER TABLE dbo.orders ADD {column} {definition};
                END
                """
            )
        cursor.execute(
            """
            IF OBJECT_ID(N’dbo.users’, N’U’) IS NULL
            BEGIN
                CREATE TABLE dbo.users (
                    user_id INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
                    username NVARCHAR(100) NOT NULL UNIQUE,
```

```
                    password_hash NVARCHAR(255) NOT NULL,
                    role NVARCHAR(50) NOT NULL,
                    full_name NVARCHAR(255) NULL,
                    email NVARCHAR(255) NULL,
                    address NVARCHAR(500) NULL
                );
            END
            """
        )
        conn.commit()
    seed_default_admin()


def hash_password(password):
    salt = os.urandom(16)
    password_hash = hashlib.pbkdf2_hmac(
        "sha256",
        password.encode("utf-8"),
        salt,
        100000,
    )
    return f"{salt.hex()}:{password_hash.hex()}"


def verify_password(password, stored_hash):
    try:
        salt_hex, hash_hex = stored_hash.split(":", 1)
        expected_hash = bytes.fromhex(hash_hex)
        actual_hash = hashlib.pbkdf2_hmac(
            "sha256",
            password.encode("utf-8"),
            bytes.fromhex(salt_hex),
            100000,
        )
        return hmac.compare_digest(actual_hash, expected_hash)
    except ValueError:
        return False


def seed_default_admin():
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute(
            "SELECT 1 FROM dbo.users WHERE username = ?;",
            DEFAULT_ADMIN_USERNAME,
        )
        if cursor.fetchone():
            return

        cursor.execute(
            """
            INSERT INTO dbo.users
                (username, password_hash, role, full_name, email)
            VALUES (?, ?, ?, ?, ?);
            """,
            DEFAULT_ADMIN_USERNAME,
            hash_password(DEFAULT_ADMIN_PASSWORD),
            DEFAULT_ADMIN_ROLE,
            "Default Admin",
            "admin@example.com",
        )
        conn.commit()


def authenticate_user(username, password):
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute(
            """
            SELECT username, password_hash, role, full_name, email, address
            FROM dbo.users
            WHERE username = ?;
            """,
            username,
        )
        row = cursor.fetchone()
        if not row or not verify_password(password, row.password_hash):
            return None
        return {
            "username": row.username,
            "role": row.role,
            "full_name": row.full_name,
```

```
            "email": row.email,
            "address": row.address,
        }


def _is_table_empty():
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute("SELECT COUNT(1) FROM dbo.orders;")
        count = cursor.fetchone()[0]
        return count == 0


def seed_from_csv(csv_path="orders.csv"):
    if not os.path.exists(csv_path):
        return
    if not _is_table_empty():
        return

    rows = []
    with open(csv_path, mode="r") as file:
        reader = csv.DictReader(file)
        for row in reader:
            parsed_date = _parse_order_date(row.get("expected_delivery_date"))
            rows.append(
                (
                    row["order_number"],
                    row["customer_email"],
                    row["customer_address"],
                    row.get("customer_name"),
                    row["status"],
                    row["current_location"],
                    parsed_date,
                    row.get("product_id"),
                    row.get("product_name"),
                    int(row.get("quantity") or 1),
                    row.get("unit_price") or None,
                    row.get("total_price") or None,
                )
            )

    if not rows:
        return

    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.fast_executemany = True
        cursor.executemany(
            """
            INSERT INTO dbo.orders
                (
                    order_number,
                    customer_email,
                    customer_address,
                    customer_name,
                    status,
                    current_location,
                    expected_delivery_date,
                    product_id,
                    product_name,
                    quantity,
                    unit_price,
                    total_price
                )
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?);
            """,
            rows,
        )
        conn.commit()


def _parse_order_date(raw_date):
    raw_date = (raw_date or "").strip()
    if not raw_date or raw_date.upper() == "N/A":
        return None
    return datetime.strptime(raw_date, "%d/%m/%Y").date()


def fetch_order_info(order_number):
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute(
```

```
            """
            SELECT
                order_number,
                customer_email,
                customer_address,
                customer_name,
                status,
                current_location,
                expected_delivery_date,
                product_id,
                product_name,
                quantity,
                unit_price,
                total_price
            FROM dbo.orders
            WHERE order_number = ?;
            """,
            order_number,
        )
        row = cursor.fetchone()
        if not row:
            return None
        return {
            "order_number": row.order_number,
            "customer_email": row.customer_email,
            "customer_address": row.customer_address,
            "customer_name": getattr(row, "customer_name", None),
            "status": row.status,
            "current_location": row.current_location,
            "expected_delivery_date": str(row.expected_delivery_date),
            "product_id": getattr(row, "product_id", None),
            "product_name": getattr(row, "product_name", None),
            "quantity": getattr(row, "quantity", None),
            "unit_price": getattr(row, "unit_price", None),
            "total_price": getattr(row, "total_price", None),
        }


def verify_customer_email(order_number, customer_email):
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute(
            """
            SELECT 1
            FROM dbo.orders
            WHERE order_number = ? AND customer_email = ?;
            """,
            order_number,
            customer_email,
        )
        return cursor.fetchone() is not None


def fetch_customer_address(email):
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute(
            """
            SELECT customer_address
            FROM dbo.orders
            WHERE customer_email = ?;
            """,
            email,
        )
        row = cursor.fetchone()
        return row.customer_address if row else None


def update_address(order_number, new_address):
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute(
            """
            UPDATE dbo.orders
            SET customer_address = ?
            WHERE order_number = ?;
            """,
            new_address,
            order_number,
        )
        conn.commit()
```

```
def update_order_status(order_number, status, current_location=None):
    with get_connection() as conn:
        cursor = conn.cursor()
        if current_location:
            cursor.execute(
                """
                UPDATE dbo.orders
                SET status = ?, current_location = ?
                WHERE order_number = ?;
                """,
                status,
                current_location,
                order_number,
            )
        else:
            cursor.execute(
                """
                UPDATE dbo.orders
                SET status = ?
                WHERE order_number = ?;
                """,
                status,
                order_number,
            )
        updated_count = cursor.rowcount
        conn.commit()
        return updated_count > 0


def delete_order(order_number):
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute(
            """
            DELETE FROM dbo.orders
            WHERE order_number = ?;
            """,
            order_number,
        )
        deleted_count = cursor.rowcount
        conn.commit()
        return deleted_count > 0


def generate_order_number():
    with get_connection() as conn:
        cursor = conn.cursor()
        return _generate_order_number(cursor)


def _generate_order_number(cursor):
    pattern = f"{ORDER_NUMBER_PREFIX}%"
    regex = re.compile(rf"^{re.escape(ORDER_NUMBER_PREFIX)}(\d+)$")

    cursor.execute(
        """
        SELECT order_number
        FROM dbo.orders
        WHERE order_number LIKE ?;
        """,
        pattern,
    )

    highest_number = 0
    for row in cursor.fetchall():
        match = regex.match(row.order_number)
        if match:
            highest_number = max(highest_number, int(match.group(1)))

    next_number = highest_number + 1
    return f"{ORDER_NUMBER_PREFIX}{next_number:0{ORDER_NUMBER_DIGITS}d}"


def create_order(
    customer_email,
    customer_address,
    status,
    current_location,
    expected_delivery_date=None,
    customer_name=None,
    product_id=None,
```

```
    product_name=None,
    quantity=1,
    unit_price=None,
    total_price=None,
    order_number=None,
):
    parsed_date = _parse_order_date(expected_delivery_date)
    with get_connection() as conn:
        cursor = conn.cursor()
        order_number = order_number or _generate_order_number(cursor)
        cursor.execute(
            "SELECT 1 FROM dbo.orders WHERE order_number = ?;",
            order_number,
        )
        if cursor.fetchone():
            return False, "Order number already exists."

        cursor.execute(
            """
            INSERT INTO dbo.orders
                (
                    order_number,
                    customer_email,
                    customer_address,
                    customer_name,
                    status,
                    current_location,
                    expected_delivery_date,
                    product_id,
                    product_name,
                    quantity,
                    unit_price,
                    total_price
                )
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?);
            """,
            order_number,
            customer_email,
            customer_address,
            customer_name,
            status,
            current_location,
            parsed_date,
            product_id,
            product_name,
            quantity,
            unit_price,
            total_price,
        )
        conn.commit()
        return True, f"Order created. Order number: {order_number}"
```

train.py

```
import json

import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, Dataset

from model import NeuralNet
from nltk_utils import bag_of_words, lemmatize, tokenize


def load_data(filename):
    with open(filename, 'r') as f:
        intents = json.load(f)

    all_words = []
    tags = []
    xy = []

    for intent in intents['intents']:
        tag = intent['tag']
        tags.append(tag)
        for pattern in intent['queries']:
            w = tokenize(pattern)
            all_words.extend(w)
            xy.append((w, tag))

    ignore_words = ['?', '.', '!', 'is', 'the', 'are', 'what', 'can']
    all_words = [lemmatize(w) for w in all_words if w not in ignore_words]
    all_words = sorted(set(all_words))
    tags = sorted(set(tags))

    return all_words, tags, xy


def create_training_data(all_words, tags, xy):
    X_train = []
    y_train = []

    for pattern_sentence, tag in xy:
        bag = bag_of_words(pattern_sentence, all_words)
        X_train.append(bag)
        label = tags.index(tag)
        y_train.append(label)

    return np.array(X_train), np.array(y_train)


class ChatDataset(Dataset):
    def __init__(self, X, y):
        self.x_data = torch.Tensor(X).float()
        self.y_data = torch.Tensor(y).long()
        self.n_samples = len(self.x_data)

    def __getitem__(self, index):
        return self.x_data[index], self.y_data[index]

    def __len__(self):
        return self.n_samples


def set_hyperparameters(X_train, tags):
    input_size = len(X_train[0])
    hidden_size = 16
    output_size = len(tags)
    num_epochs = 1000
    batch_size = 8
    learning_rate = 0.001

    return input_size, hidden_size, output_size, num_epochs, batch_size, learning_rate


def train_model(
    dataset,
    input_size,
    hidden_size,
    output_size,
    num_epochs,
    batch_size,
```

```
    learning_rate,
):
    train_loader = DataLoader(dataset=dataset, batch_size=batch_size, shuffle=True)
    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
    model = NeuralNet(input_size, hidden_size, output_size).to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)

    for epoch in range(num_epochs):
        for words, labels in train_loader:
            words = words.to(device)
            labels = labels.to(device)

            outputs = model(words)
            loss = criterion(outputs, labels)

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

        if (epoch + 1) % 100 == 0:
            print(f'Epoch [{epoch + 1}/{num_epochs}], Loss: {loss.item():.4f}')

    print(f'Final loss: {loss.item():.4f}')
    return model


def save_model(
    model,
    input_size,
    hidden_size,
    output_size,
    all_words,
    tags,
    filename,
):
    data = {
        "model_state": model.state_dict(),
        "input_size": input_size,
        "hidden_size": hidden_size,
        "output_size": output_size,
        "all_words": all_words,
        "tags": tags,
    }
    torch.save(data, filename)
    print(f'Training complete. Model saved to {filename}')


if __name__ == "__main__":
    all_words, tags, xy = load_data('intents.json')

    print(f"Number of queries: {len(xy)}")
    print(f"Number of tags: {len(tags)}")
    print("\nUnique lemmatized Words:")
    print(" ".join(all_words))

    X_train, y_train = create_training_data(all_words, tags, xy)
    (
        input_size,
        hidden_size,
        output_size,
        num_epochs,
        batch_size,
        learning_rate,
    ) = set_hyperparameters(X_train, tags)
    dataset = ChatDataset(X_train, y_train)
    model = train_model(
        dataset,
        input_size,
        hidden_size,
        output_size,
        num_epochs,
        batch_size,
        learning_rate,
    )
    save_model(
        model,
        input_size,
        hidden_size,
        output_size,
        all_words,
        tags,
```

```
    'data.pth',
)
```

model.py

```
import torch.nn as nn


class NeuralNet(nn.Module):
    def __init__(self, input_size, hidden_size, num_classes):
        super(NeuralNet, self).__init__()
        self.l1 = nn.Linear(input_size, hidden_size)
        self.l2 = nn.Linear(hidden_size, hidden_size)
        self.l3 = nn.Linear(hidden_size, num_classes)
        self.relu = nn.ReLU()

    def forward(self, x):
        out = self.l1(x)
        out = self.relu(out)
        out = self.l2(out)
        out = self.relu(out)
        out = self.l3(out)

        return out
```

nltk_utils.py

```
import numpy as np  # type: ignore[import-untyped]
import nltk  # type: ignore[import-untyped]
from nltk.stem.porter import PorterStemmer  # type: ignore[import-untyped]
from nltk.stem import WordNetLemmatizer  # type: ignore[import-untyped]

stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()


def tokenize(sentence):
    """
    split sentence into array of words/tokens
    a token can be a word or punctuation character, or number
    """
    return nltk.word_tokenize(sentence)


def stem(word):
    """
    stemming = find the root form of the word
    examples:
    words = ["organize", "organizes", "organizing"]
    words = [stem(w) for w in words]
    -> ["organ", "organ", "organ"]
    """
    return stemmer.stem(word.lower())


def lemmatize(word):
    """
    lemmatization = get base form of the word
    examples:
    words = ["organize", "organizes", "organizing"]
    words = [lemmatize(w) for w in words]
    -> ["organize", "organize", "organizing"]
    """
    return lemmatizer.lemmatize(word.lower())


def bag_of_words(tokenized_sentence, words):
    """
    return bag of words array:
    1 for each known word that exists in the sentence, 0 otherwise
    example:
    sentence = ["hello", "how", "are", "you"]
    words = ["hi", "hello", "I", "you", "bye", "thank", "cool"]
    bog   = [  0 ,    1 ,    0 ,   1 ,    0 ,    0 ,      0]
    """
    sentence_words = [stem(word) for word in tokenized_sentence]

    bag = np.zeros(len(words), dtype=np.float32)
    for idx, w in enumerate(words):
        if w in sentence_words:
            bag[idx] = 1

    return bag
```

intents.json


```
{
  "intents": [
  {
    "tag": "greeting",
    "queries": ["Hi", "Hey", "How are you", "Is anyone there?", "Hello", "Good day"],
    "responses": ["Hey :-)", "Hello, thanks for visiting", "Hi there, what can I do for you?",
"Hi there, how can I help?"]
  },
  {
    "tag": "goodbye",
    "queries": ["Bye", "See you later", "Goodbye", "Farewell", "Take care"],
    "responses": ["See you later, thanks for visiting", "Have a nice day", "Bye! Come back again
 soon.", "Goodbye! Stay safe."]
  },
  {
    "tag": "thanks",
    "queries": ["Thanks", "Thank you", "That’s helpful", "Thank’s a lot!", "I appreciate it"],
    "responses": ["Happy to help!", "Any time!", "My pleasure", "You’re welcome!", "Glad I could
 assist!"]
  },
  {
    "tag": "items",
    "queries": [
      "Which items do you have?",
      "What kinds of items are there?",
      "What do you sell?",
      "What products do you have?",
      "What products do you offer?",
      "Can you tell me about your range of products?",
      "What kind of items do you sell?",
      "Do you have a catalog or list of products?",
      "What are your best-selling products?",
      "Can you describe your product lineup?",
      "What types of services do you provide?",
      "What’s in stock at your store?",
      "Can you recommend some products you offer?",
      "what do you offer?",
      "show products",
      "show me products",
      "list products",
      "show product catalog",
      "what products are available?",
      "I want to see products"
    ],
    "responses": [
      "We offer coffee mugs in red, white, and blue colors. They’re perfect for enjoying your
favorite hot beverages!",
      "We have a variety of coffee mugs available in red, white, and blue. They come in
different sizes and designs to suit your preferences.",
      "Our coffee mugs are available in classic colors like red, white, and blue. They’re
durable, stylish, and great for everyday use.",
      "We sell high-quality coffee mugs in red, white, and blue colors. They’re microwave and
dishwasher safe for your convenience.",
      "You’ll love our selection of coffee mugs! We have them in vibrant red, crisp white, and
patriotic blue colors.",
      "Our coffee mugs come in a trio of colors: red, white, and blue. They make excellent gifts
 or additions to your own collection.",
      "We offer coffee mugs in red, white, and blue shades. They’re made from premium materials
and designed for both style and functionality.",
      "Whether you prefer red, white, or blue, we’ve got you covered with our range of coffee
mugs. They’re both stylish and practical for everyday use.",
      "Our coffee mugs are available in three classic colors: red, white, and blue. They’re
perfect for enjoying a cup of coffee or tea any time of the day.",
      "We have a great selection of coffee mugs in red, white, and blue. They’re ideal for home
or office use and make a wonderful addition to any kitchen."
    ]
  },
  {
    "tag": "payments",
    "queries": [
      "Do you take credit cards?",
      "Do you accept Mastercard?",
      "Can I pay with Paypal?",
      "Are you cash only?",
      "What payment methods do you accept?",
      "How can I pay?",
      "Accepted payment options?",
      "Payment methods?",
```

```
      "Can I use Apple Pay?",
      "Do you accept online payments?",
      "Do you support Visa payments?",
      "Can I use American Express for payment?",
      "Is Google Pay accepted?",
      "Do you offer installment payments?",
      "Are debit cards accepted?",
      "Can I pay with a bank transfer?",
      "Do you accept mobile payments?",
      "Are there any additional fees for online payments?",
      "Is PayPal the only online payment option?",
      "Do you offer payment plans or financing options?"
    ],
    "responses": [
      "We accept VISA, Mastercard, and Paypal",
      "We accept most major credit cards, and Paypal",
      "We accept major cards like Visa, Mastercard, and American Express",
      "You can use major cards like Visa, Mastercard, and American Express for seamless
transactions",
      "Most credits cards, Visa and Mastercards are accepted",
      "You may use Paypal or any major credit card, Visa and Mastercard for purchase",
      "We accept major card brands like Visa, Mastercard, and American Express, allowing for
easy and secure transactions.",
      "Currently, we accept VISA, Mastercard, American Express, and Paypal. We also offer secure
 online payment options.",
      "Yes, we accept Apple Pay, along with major credit cards and Paypal.",
      "We accept online payments through our secure checkout process. You can use major credit
cards, Paypal, and Apple Pay."
    ]
  },
  {
    "tag": "delivery",
    "queries": [
      "How long does delivery take?",
      "How long does shipping take?",
      "When do I get my delivery?",
      "How can I track my order?",
      "Shipping details?",
      "Delivery time?",
      "Tracking my package?",
      "How long does it take",
      "What is the estimated delivery time?",
      "When will my package arrive?",
      "Can I check the shipping status?",
      "How can I monitor my shipment?",
      "Do you provide tracking information?",
      "When should I expect my order?",
      "How can I find my order status?",
      "Is express shipping available?",
      "deliver",
      "how fast can you deliver?"
    ],
    "responses": [
      "Delivery takes 2-4 days",
      "Shipping takes 2-4 days",
      "Your order will arrive within 2-4 business days.",
      "The estimated delivery time is 2-4 business days. You will receive a tracking number to
monitor the progress of your order."
    ]
      },
      {
        "tag": "shippable_cities",
        "queries": [
          "Do you ship to Beijing?",
          "Can you deliver to Shanghai?",
          "Is shipping available in Guangzhou?",
          "Can I receive my order in Shenzhen?",
          "Do you offer delivery to Hangzhou?",
          "shanghai",
          "guangzhou",
          "shenzhen"
        ],
        "responses": [
          "Yes, we ship to your city",
          "Certainly! We deliver to your city.",
          "Shipping is available in your city.",
          "You can receive your order in your city.",
          "Yes, we offer delivery to your city."
        ]
      },
      {
        "tag": "non_shippable_cities",
```

```
        "queries": [
          "Can I get my order in Lhasa?",
          "Do you ship to Urumqi?",
          "Is delivery available in Lanzhou?",
          "Can I receive my purchase in Kunming?",
          "Do you offer shipping to Hohhot?"
        ],
        "responses": [
          "I’m sorry, we currently do not ship to Lto your city.",
          "Unfortunately, we do not deliver to your city.",
          "Delivery is not available in your city.",
          "We do not offer shipping to your city.",
          "Sorry, we currently do not ship to your city."
        ]
      },
      {
        "tag": "funny",
        "queries": ["Tell me a joke!", "Tell me something funny!", "Do you know a joke?", "Got
any jokes?"],
        "responses": ["Why did the hipster burn his mouth? He drank the coffee before it was
cool.", "What did the buffalo say when his son left for college? Bison."]
      },
      {"tag": "update_address",
      "queries": [
      "I don’t know how to update my delivery address",
      " I do not know how to submit a different delivery address",
      "how to edit the address?",
      "where can I set the delivery address up?",
      "I do not know how to submit a different delivery address",
      "I need assistance with a shipping address update",
      "I’m trying to set up another shipping address",
      "want help updating the address",
      "can you help me to set a shipping address up?",
      " I need to submit the new delivery address",
      "How can I change my shipping address?",
      "I want to update my delivery details",
      "Can I modify the address for my order?",
      "Where can I change my delivery address?",
      "I need to correct my shipping information",
      "How do I edit my delivery address?",
      "I want to set a new shipping address",
      "Can you guide me through updating my address?",
      "I need to alter the delivery address",
      "How to update the shipping details?",
      "how to update it?",
      "update"
       ],
        "responses": [
      "I can help you with that.",
      "Sure, let me assist you with that",
      "I can help you change it"

    ]
    },
    {"tag": "sizes",
    "queries": [
      "What sizes are available?",
      "Do you offer different sizes?",
      "What size are the mugs?",
      "Can I choose the size of the coffee mugs?",
      "Are the mugs available in different sizes?",
      "in what color"
    ],
    "responses": [
      "Our coffee mugs come in standard sizes that are perfect for everyday use.",
      "We offer coffee mugs in various sizes to suit your needs.",
      "You can choose from our range of coffee mugs available in different sizes.",
      "Our coffee mugs are available in standard sizes suitable for all preferences.",
      "We provide coffee mugs in sizes that cater to both small and large preferences."
    ]
  },
  {
    "tag": "material",
    "queries": [
      "What are the mugs made of?",
      "Are the mugs ceramic or porcelain?",
      "What material are the coffee mugs?",
      "Are the mugs dishwasher safe?",
      "Can you tell me about the material of the mugs?",
      "What kind of material are the mugs made from?",
      "Are the mugs made of high-quality material?",
      "What are the mugs constructed of?",
```

"Are the mugs microwave-safe?",
"Can you provide information about the mug material?",
"What is the composition of the coffee mugs?",
"Are the mugs eco-friendly?",
"Do the mugs contain any harmful substances?",
"What is the durability of the mug material?",
"Are the mugs lead-free?"
],
"responses": [
"Our coffee mugs are made of high-quality ceramic material.",
"We offer coffee mugs crafted from durable and dishwasher-safe ceramic.",
"Our coffee mugs are made from premium ceramic material, ensuring longevity and durability.",
"The coffee mugs we sell are both microwave and dishwasher safe, made from quality ceramic.",
"Our coffee mugs are crafted from ceramic material, making them both stylish and durable."
]
},
{
"tag": "care_instructions",
"queries": [
"How do I care for the mugs?",
"Are the mugs microwave safe?",
"Can I put the mugs in the dishwasher?",
"Do the mugs require special care?",
"Can you provide care instructions for the mugs?"
],
"responses": [
"To ensure the longevity of your coffee mugs, we recommend hand washing them.",
"Our coffee mugs are microwave and dishwasher safe for your convenience.",
"We recommend hand washing the mugs to maintain their quality and appearance.",
"The mugs are both microwave and dishwasher safe, making them easy to care for.",
"For best results, we suggest hand washing the mugs to preserve their vibrant colors and design."
]
},
{
"tag": "gifts",
"queries": [
"Are the mugs suitable as gifts?",
"Can I gift the coffee mugs?",
"Do you offer gift wrapping?",
"Are gift options available for the mugs?",
"Can the mugs be shipped as gifts?",
"Are the mugs gift-ready?",
"Can I purchase the mugs as presents?",
"Do you provide gift messages with the mugs?",
"Are there any gift sets available with the mugs?",
"Can I personalize the mugs for gifting?",
"Do you offer gift certificates for the mugs?",
"Are the mugs suitable for special occasions?",
"Can I include a gift note with the mugs?",
"Do you have any special packaging for gifting?",
"Are there any discounts or promotions for gifting mugs?"
],
"responses": [
"Our coffee mugs make excellent gifts for any occasion, available in various colors to suit everyone’s taste.",
"You can gift our stylish coffee mugs, available in red, white, and blue colors.",
"We offer gift wrapping options for our coffee mugs to make your gift extra special.",
"Our coffee mugs come in a beautiful packaging, making them perfect gifts for friends and family.",
"You can easily gift our coffee mugs, and we provide gift wrapping options to enhance the presentation."
]
},
{
"tag": "user_preferences_personalization",
"queries": [
"Can I customize this?",
"Do you have this in my size/color?",
"Can I personalize my order?",
"Can I choose a different color?",
"Is customization available?",
"Do you offer personalization options?",
"Can I select a custom design?",
"Is there an option to customize the mug?",
"Can I choose my preferred design/color?",
"Do you have a customization service?",
"Can I personalize the mug with a name or message?",
"Is there a way to customize the product?",
"Do you offer color choices for the mugs?",

```
      "Can I add a custom logo or image?",
      "Is it possible to personalize the order details?",
      "customize",
      "personalize"
    ],
    "responses": [
      "Certainly! You can customize your order as per your preferences.",
      "Yes, we have a range of sizes and colors available.",
      "Absolutely, you can personalize your order to suit your preferences.",
      "Yes, you can choose from our available colors.",
      "Yes, customization options are available for your order."
    ]
  },
  {
    "tag": "negation_reversal",
    "queries": [
      "I changed my mind.",
      "Not interested anymore.",
      "Cancel that.",
      "I'll pass on that.",
      "I've decided against it."
    ],
    "responses": [
      "No problem! Let me know if you change your mind again.",
      "Alright, let me know if you reconsider.",
      "Sure, I've cancelled that for you.",
      "Alright, let me know if you need anything else.",
      "Okay, feel free to reach out if you change your mind."
    ]
  },
  {
    "tag": "order_status",
    "queries": [
    "Where is my order?",
    "What's the status of my order?",
    "Has my order been shipped?",
    "When will my order arrive?",
    "Track my order",
    "track order",
    "i would like to track my order",
    "what is its location?",
    "can you check my order?"],
    "responses": [
      "Let me check that for you.",
      "I can help with that.",
      "Sure."]
},
{
    "tag": "returns_refunds",
    "queries": [
      "Can I return my mug?",
      "What is your return policy?",
      "How do I get a refund?",
      "Can I exchange my mug?",
      "Return process",
      "refund"],
    "responses": [
      "Our return policy allows returns within 30 days of purchase. Please visit our Returns &
Refunds page for more details.",
      "You can request a refund or exchange within 30 days of purchase. Please visit our Returns
 & Refunds page for instructions."]
},
{
    "tag": "promotions_discounts",
    "queries": [
      "Do you have any promotions?",
      "Are there any discounts available?",
      "Can I use a coupon code?",
      "Special offers",
      "Discounts on mugs",
      "discounts"],
    "responses": [
      "Yes, we currently have a 10% discount on selected mugs. Use code MUG10 at checkout!",
      "Sign up for our newsletter to receive exclusive discounts and promotions."]
},
{
    "tag": "product_availability",
    "queries": [
    "Is this mug in stock?",
    "When will this mug be available?",
    "Product availability",
    "Out of stock mugs",
```

```
    "Can I pre-order a mug?",
    "stock"],
    "responses": [
    "The mug you're interested in is currently in stock and ready to ship!",
    "Unfortunately, that mug is currently out of stock. You can sign up to be notified when it's
 back in stock."]
},
{
    "tag": "customer_service",
    "queries": [
      "How can I contact customer service?",
      "I need help from customer service",
      "Contact support",
      "Customer service hours",
      "Live chat support"],
    "responses": [
      "You can reach our customer service team at support@coffeemugs.com ",
      "Our customer service team is available from 9am to 6pm EST, Monday to Friday."]
},
{
    "tag": "product_care",
    "queries": [
      "How do I care for my mug?",
      "Mug care instructions",
      "Can I put my mug in the dishwasher?",
      "Cleaning my mug", "Care tips for mugs"],
    "responses": [
      "To prolong the life of your mug, we recommend hand washing it with mild soap and avoiding
 abrasive cleaners.",
      "Our mugs are dishwasher safe, but hand washing is recommended for best results."]
},
{
    "tag": "warranty_repairs",
    "queries": [
      "Is there a warranty on my mug?",
      "How do I get my mug repaired?",
      "Mug warranty information",
      "Broken mug",
      "Defective mug",
      "warranty",
      "do you have warranty"],

    "responses": [
      "All our mugs come with a 1-year warranty against manufacturing defects. Please contact
our customer service for repair or replacement options.",
      "We apologize for the inconvenience. Please contact our customer service team for
assistance with your defective mug."]
},
{
    "tag": "shipping_costs_time",
    "queries": [
      "How much does shipping cost?",
      "When will my mug arrive?",
      "Shipping time for mugs",
      "Express shipping options",
      "International shipping",
      "what is the delivery fee?",
      "shipping fee",
      "delivery fee",
      "shipping cost",
      "cost",
      "what is the cost of shipping?"],
    "responses": [
      "Standard shipping is $5, and it usually takes 2-4 business days for your mug to arrive."
      ]
},
{
    "tag": "bulk_orders",
    "queries": [
      "Can I order mugs in bulk?",
      "Wholesale purchasing",
      "Bulk discount on mugs",
      "Large orders of mugs",
      "Corporate gifts",
      "bulk"],
    "responses": [
      "Yes, we offer bulk ordering and wholesale purchasing options. Please contact our sales
team for more information and special pricing.",
      "Looking to place a large order for corporate gifts? We've got you covered! Contact our
sales team for personalized assistance."]
},
{
```

```
    "tag": "account_profile",
    "queries": [
      "How do I create an account?",
      "Login issues",
      "Update my profile",
      "Forgot password",
      "Account settings",
      "account",
      "profile",
      "how do i sign up?",
      "how do i make an account?",
      "how do i make a profile"],
    "responses": [
      "You can easily create an account by clicking on the 'Sign Up' button at the top right
corner of our website. For login issues or password resets, please visit the 'Account Settings'
page.",
      "If you're having trouble logging in or need to reset your password, please visit our
Account Settings page or contact customer service for assistance."]
},
{
    "tag": "product_reviews_ratings",
    "queries": [
      "Can I see product reviews?",
      "Mug ratings",
      "Customer reviews",
      "Product recommendations",
      "Top-rated mugs",
      "reviews"],
    "responses": [
      "You can find customer reviews and ratings on each product page. We also feature top-rated
 mugs in our Best Sellers section.",
      "Customer feedback is important to us! Check out our product pages for reviews, ratings,
and recommendations."]
},
{
    "tag": "loyalty_programs",
    "queries": [
      "Do you have a loyalty program?",
      "Earning rewards",
      "Membership benefits",
      "Loyalty points",
      "Joining the rewards program"],
    "responses": [
      "Yes, we have a loyalty program where you can earn points on every purchase. Sign up today
 to start earning rewards and enjoy exclusive member benefits!",
      "Our loyalty program rewards you with points on every purchase, which can be redeemed for
discounts on future orders. Join today and start saving!"]
},
{
    "tag": "privacy_security",
    "queries": [
      "What is your privacy policy?",
      "Is my data secure?",
      "Data protection",
      "Account security",
      "Privacy settings"],
    "responses": [
    "We take your privacy seriously. You can view our full privacy policy on our website to
learn how we collect, use, and protect your personal information.",
    "Protecting your data is our top priority. We use industry-standard encryption and security
measures to safeguard your information."
  ]
},
{
    "tag": "technical_support",
    "queries": [
      "I'm having trouble with the website",
      "Technical issues",
      "Website not working",
      "Troubleshooting",
      "Error messages"
    ],
    "responses": [
      "We apologize for any inconvenience you're experiencing. Please try clearing your
browser's cache and cookies or using a different browser. If the issue persists, please contact
our technical department"]
},
{
    "tag": "create_order",
    "queries": [
      "create order",
      "place order",
```

```
      "new order",
      "make an order",
      "I want to place an order",
      "create a new order",
      "add order",
      "order a product",
      "buy a product",
      "I want to buy a product"
    ],
    "responses": [
      "Sure, please fill in the order details in the form below.",
      "I can help with that. Please provide the order details.",
      "Got it. Use the order form to create a new order."
    ]
  },
  {
    "tag": "cancel_order",
    "queries": [
      "cancel order",
      "delete order",
      "remove order",
      "cancel my order",
      "delete my order",
      "remove my order",
      "I want to cancel my order",
      "I want to delete my order",
      "Can you cancel my order?",
      "Can you delete my order?",
      "Cancel an existing order",
      "Delete an existing order"
    ],
    "responses": [
      "Sure, I can help cancel your order.",
      "I can help with that. Please provide the order details.",
      "Okay, I will verify your order before cancelling it."
    ]
  },
  {
    "tag": "admin_login",
    "queries": [
      "admin login",
      "login as admin",
      "admin",
      "I want to login as admin"
    ],
    "responses": [
      "Please enter your admin credentials."
    ]
  },
  {
    "tag": "recommendations",
    "queries": [
      "Recommend similar products",
      "Can you recommend a mug for me?",
      "Suggest products based on what I viewed",
      "What should I buy?",
      "Show me personalized recommendations",
      "Recommend something like P001",
      "Suggest a similar mug",
      "What products are similar to this?",
      "Can you suggest items for my cart?",
      "Best match for me"
    ],
    "responses": [
      "I can recommend products based on your browsing and cart activity.",
      "Here are some products that match your interests.",
      "I found a few similar products for you."
    ]
  },
  {
    "tag": "product_popularity_analysis",
    "queries": [
      "Show product popularity",
      "What are the most popular products?",
      "Show me the best selling mugs",
      "Top selling products",
      "Product popularity analysis",
      "Sales analysis",
      "Show analytics"
    ],
    "responses": [
      "I can show product popularity using order history.",
```

```
      "Here is the sales-based product popularity analysis."
    ]
},
{
    "tag": "market_basket_analysis",
    "queries": [
      "Market basket analysis",
      "Run Apriori analysis",
      "Which products are frequently bought together?",
      "Show product pairs",
      "Basket analysis",
      "Frequently bought together"
    ],
    "responses": [
      "I can analyze product pairs bought by the same customers.",
      "Here is the market basket analysis from order history."
    ]
},
{
    "tag": "product_category_prediction",
    "queries": [
      "Predict category for P001",
      "What category is P004?",
      "Which category does this product belong to?",
      "Classify product Classic Red Mug",
      "Product category prediction",
      "Predict category for insulated travel mug",
      "What category is a gift mug set?",
      "Classify this product as a category"
    ],
    "responses": [
      "I can predict the product category using product text features.",
      "I will classify the product into a category."
    ]
},
{
    "tag": "update_order_status",
    "queries": [
      "update order status",
      "change order status",
      "set order status",
      "update order location",
      "change order location",
      "mark order as shipped",
      "mark order as delivered"
    ],
    "responses": [
      "Admin access is required to update order status."
    ]
}


]
}
```

products.csv

```
product_id,name,category,color,size,material,price,stock,description
P001,Classic Red Mug,Coffee Mug,Red,350ml,Ceramic,12.99,25,Durable everyday ceramic mug in red
P002,Classic White Mug,Coffee Mug,White,350ml,Ceramic,11.99,40,Minimal white ceramic mug for
coffee or tea
P003,Classic Blue Mug,Coffee Mug,Blue,350ml,Ceramic,12.99,30,Blue ceramic mug with dishwasher-
safe finish
P004,Large Travel Mug,Travel Mug,Black,500ml,Stainless Steel,18.99,15,Insulated travel mug for
hot drinks
P005,Gift Mug Set,Gift Set,Mixed,3 x 350ml,Ceramic,34.99,10,Three-piece mug gift set with red
white and blue mugs
P006,Matte Black Mug,Coffee Mug,Black,400ml,Ceramic,14.99,22,Modern matte ceramic mug for coffee
 and tea
P007,Pastel Pink Mug,Coffee Mug,Pink,300ml,Ceramic,13.49,18,Small pastel ceramic mug with smooth
 finish
P008,Speckled Stoneware Mug,Coffee Mug,Grey,450ml,Stoneware,16.99,14,Handcrafted stoneware mug
with speckled glaze
P009,Glass Latte Mug,Latte Mug,Clear,300ml,Glass,10.99,35,Transparent glass mug for latte
cappuccino and tea
P010,Double Wall Espresso Cup,Espresso Cup,Clear,100ml,Glass,9.99,28,Double wall glass espresso
cup for hot drinks
P011,Compact Travel Tumbler,Travel Mug,Silver,350ml,Stainless Steel,16.99,20,Compact insulated
stainless steel travel tumbler
P012,Premium Thermal Travel Mug,Travel Mug,Navy,600ml,Stainless Steel,22.99,12,Large thermal
travel mug with leak resistant lid
P013,Bamboo Lid Mug,Eco Mug,White,380ml,Ceramic,15.99,16,Ceramic eco mug with reusable bamboo
lid
P014,Reusable Wheat Straw Cup,Eco Mug,Beige,420ml,Wheat Straw,8.99,45,Lightweight reusable eco
cup made from wheat straw
P015,Holiday Gift Mug Set,Gift Set,Mixed,4 x 300ml,Ceramic,42.99,8,Four-piece holiday ceramic
mug gift set
```

orders.csv

```
order_number,status,current_location,expected_delivery_date,customer_address,customer_name,custo
mer_email,product_id,product_name,quantity,unit_price,total_price
1122,Shipped,Shanghai,02/05/2024,Mandalay,John Doe,john@gmail.com,P001,Classic Red
Mug,2,12.99,25.98
1123,Delivered,Beijing,01/05/2024,Shenzhen,Alice Smith,alice@gmail.com,P002,Classic White
Mug,1,11.99,11.99
1124,Processing,Guangzhou,05/05/2024,Shanghai,Bob Johnson,bob@gmail.com,P003,Classic Blue
Mug,3,12.99,38.97
1125,Shipped,Shenzhen,03/05/2024,Beijing,Emma Brown,emma@gmail.com,P005,Gift Mug
Set,1,34.99,34.99
1126,Pending,Hangzhou,06/05/2024,Chengdu,David Wilson,david@example.com,P004,Large Travel
Mug,1,18.99,18.99
1127,Delivered,Chengdu,30/04/2024,Hangzhou,Sophia Miller,sophia@example.com,P001,Classic Red
Mug,1,12.99,12.99
1128,Cancelled,Wuhan,N/A,Shanghai,Michael Lee,michael@example.com,P002,Classic White
Mug,1,11.99,11.99
1129,Shipped,Nanjing,04/05/2024,Guangzhou,Olivia Taylor,olivia@example.com,P003,Classic Blue
Mug,2,12.99,25.98
1130,Processing,Shenzhen,07/05/2024,Beijing,Daniel Martinez,daniel@example.com,P004,Large Travel
 Mug,1,18.99,18.99
1131,Pending,Beijing,08/05/2024,Shenzhen,Emily Garcia,emily@example.com,P005,Gift Mug
Set,1,34.99,34.99
1132,Delivered,Guangzhou,29/04/2024,Shanghai,William Anderson,william@example.com,P001,Classic
Red Mug,2,12.99,25.98
1133,Shipped,Tianjin,05/05/2024,Hangzhou,James Thomas,james@example.com,P002,Classic White
Mug,2,11.99,23.98
1134,Processing,Wuhan,06/05/2024,Chengdu,Charlotte White,charlotte@example.com,P003,Classic Blue
 Mug,1,12.99,12.99
1135,Pending,Shanghai,07/05/2024,Beijing,Lucas Martin,lucas@example.com,P005,Gift Mug
Set,1,34.99,34.99
1136,Cancelled,Beijing,N/A,Shenzhen,Mia Walker,mia@example.com,P004,Large Travel
Mug,1,18.99,18.99
1137,Shipped,Nanjing,08/05/2024,Guangzhou,Ethan Harris,ethan@example.com,P001,Classic Red
Mug,3,12.99,38.97
1138,Processing,Shenzhen,09/05/2024,Shanghai,Isabella Clark,isabella@example.com,P002,Classic
White Mug,1,11.99,11.99
1139,Pending,Wuhan,10/05/2024,Chengdu,Matthew Hall,matthew@example.com,P003,Classic Blue
Mug,2,12.99,25.98
1140,Delivered,Shanghai,28/04/2024,Hangzhou,Sophia King,sophia@example.com,P005,Gift Mug
Set,1,34.99,34.99
1141,Shipped,Chongqing,11/05/2024,Beijing,Michael Green,michaelg@example.com,P004,Large Travel
Mug,2,18.99,37.98
1142,Processing,Tianjin,12/05/2024,Shenzhen,Emma Adams,emmaad@example.com,P001,Classic Red
Mug,1,12.99,12.99
1143,Pending,Guangzhou,13/05/2024,Shanghai,Benjamin Scott,benjamin@example.com,P002,Classic
White Mug,2,11.99,23.98
1144,Delivered,Wuhan,27/04/2024,Chengdu,Amelia Nelson,amelia@example.com,P003,Classic Blue
Mug,1,12.99,12.99
1145,Cancelled,Hangzhou,N/A,Beijing,Alexander Carter,alexander@example.com,P004,Large Travel
Mug,1,18.99,18.99
1146,Shipped,Chongqing,14/05/2024,Shenzhen,Harper Collins,harper@example.com,P005,Gift Mug
Set,1,34.99,34.99
1147,Processing,Beijing,15/05/2024,Guangzhou,Daniel Baker,danielb@example.com,P001,Classic Red
Mug,2,12.99,25.98
1148,Pending,Shanghai,16/05/2024,Hangzhou,Oliver Ramirez,oliver@example.com,P002,Classic White
Mug,1,11.99,11.99
1149,Delivered,Shenzhen,26/04/2024,Chengdu,Charlotte Turner,charlotte@example.com,P003,Classic
Blue Mug,2,12.99,25.98
1150,Shipped,Guangzhou,17/05/2024,Beijing,Jack White,jack@example.com,P005,Gift Mug
Set,1,34.99,34.99
1151,Delivered,Shanghai,18/05/2024,Mandalay,John Doe,john@gmail.com,P002,Classic White
Mug,1,11.99,11.99
1152,Delivered,Beijing,19/05/2024,Shenzhen,Alice Smith,alice@gmail.com,P001,Classic Red
Mug,1,12.99,12.99
1153,Shipped,Guangzhou,20/05/2024,Shanghai,Bob Johnson,bob@gmail.com,P005,Gift Mug
Set,1,34.99,34.99
1154,Delivered,Shenzhen,21/05/2024,Beijing,Emma Brown,emma@gmail.com,P003,Classic Blue
Mug,2,12.99,25.98
```

evaluation_survey.csv

```
respondent_id,chatbot_accuracy,rating_recommendations,rating_speed,rating_overall,comments
S001,5,4,5,5,Responses were clear and recommendations were useful
S002,4,4,5,4,Product suggestions were relevant
S003,4,5,4,4,Analytics answers were easy to understand
S004,5,4,4,5,Order and product features worked well
S005,4,3,5,4,Recommendation feature could include more products
```

requirements.txt

```
torch
pyodbc
nltk
numpy
```